SOFTWARE ENGINEERING / AI-ASSISTED DEVELOPMENT

Specification-Driven Development

How to make AI implementation reliable by treating specifications, not code, as the asset that lasts

Vision → Requirements → Use Cases → Domain Model → Architecture → Skills → Implementation

[AUTHOR NAME]

SOFTWARE ENGINEERING / AI-ASSISTED DEVELOPMENT

Write a specification an AI assistant implements correctly, the first time

This book teaches you to write a specification, a vision, a set of requirements, a use case, a domain model, and the architecture and implementation conventions around them, precise enough that an AI assistant builds a feature correctly on the first pass, rather than guessing at what your business needs. The same discipline runs in reverse. When a system already exists but was never documented, you can use it to recover a working specification from the code itself, business rule by business rule.

- ☐ Write a specification an AI assistant implements correctly on the first pass.
- ☐ Recover a working specification from a legacy system that never had one.
- ☐ Know what changes about your role as an engineer once implementation is mostly automated, and what doesn't.

[AUTHOR NAME] writes about specification-driven development for engineers, architects, and the managers who lead them, building on Simon Martinelli's methodology, credited throughout this book.

Self-published draft · [ISBN TBD]

Specification-Driven Development

Specification-Driven Development

How to make AI implementation reliable by treating specifications, not code, as the asset that lasts

[AUTHOR NAME]

Self-published draft

Draft edition: not for distribution.

ISBN: [ISBN TBD]

Contents

PART 3: PRACTICE AND IMPLICATIONS

BACK MATTER

Chapter page numbers are not filled in for this draft edition. The folio printed at the foot of each page is the page number.

FRONT MATTER

Preface

You already know how to write code. This book is about something you likely haven't written down yet: the decision the code is supposed to express. Two examples run through the chapters ahead, a multi-location clinic group trying to let patients book appointments online, and a retail bank trying to move a core ledger off a COBOL codebase nobody currently at the bank designed. Neither gets explained in full here. Chapter 1 opens with the clinic. Chapter 9 opens with the bank. This page only tells you they're coming.

By the last chapter, you'll be able to write a specification, a vision, a set of requirements, a use case, a domain model, and the architecture and skill conventions that surround them, precise enough that an AI assistant implements it correctly on the first pass, before review has to find what a guess got wrong. You'll also be able to run the same discipline in reverse: reading a system that was never specified, and pulling a working specification out of its behavior, because in most legacy systems, behavior is the only documentation that survived.

Some of what follows will read like process, because it is process: writing a specification asks for hours before it hands any back. The next page sorts the chapters ahead into three paths, one for an engineer writing a single feature, one for an architect keeping several teams consistent with each other, and one for a manager deciding how a team should adopt this. Pick the one that matches what you're actually deciding this week.

How to read this book

The three paths below are cuts through one argument, not three different books, and the default is to ignore them and read straight through from Chapter 1 to Chapter 11. Take a path when you have a specific decision in front of you and want the chapters that bear on it first. Nothing in a path is a substitute for the chapters it skips; it is an order of arrival.

■ PRACTITIONER

Read Chapters 1 through 11 in order. This path gives you the complete methodology, front to back, for applying it to your own feature or team.

● ARCHITECT

Read the Foreword, then Chapters 2, 4, 6, 7, and 11, then Appendix C. This path runs from vision through architecture and skills and ends at the fully worked specification template, for a reader who owns consistency across teams rather than a single feature.

◆ MANAGER

Read the Foreword, Chapter 1, Appendix A, Chapter 8, Chapter 10, and the Afterword. This path makes the case for the change and its effect on team structure and roles, without the detail of writing a specification yourself.

Foreword

In the autumn of 1969, a handful of graduate students at UCLA and a few sister sites were wiring together the first four nodes of a network called the ARPANET. Nobody involved knew yet whether any of it would hold. Among the students was a young researcher named Jon Postel. The group needed a way to write down, informally and without waiting for anyone's official blessing, how these machines were supposed to behave toward each other. They called the notes Requests for Comments. RFCs.

An RFC specified requirements: what a message must contain, what a receiver must check, what counts as an error, and what happens when one occurs. It left the actual software design open, to be decided by whoever was building the machine in front of them.

Postel took on the job of collecting and editing those notes, a role that later became known as the RFC editor, and he held it for almost three decades. It's the reason this book exists in its current form, though most of the people who will read it have never heard his name.

A specification that outlived every implementation

By 1981, two of the documents Postel edited, RFC 791 and RFC 793, set down the rules for the Internet Protocol and the Transmission Control Protocol: a description of what any implementation had to do to talk to any other implementation, detailed enough that two programmers who never met could each build a version and have them work together on the first try.

Those two documents are, at the time of this writing, older than most of the engineers who will read this book. And they have outlived every implementation ever written against them, several times over. COBOL mainframes ran an early implementation. Unix systems ran another. When personal computers arrived, they got their own. Windows, Linux, and macOS each shipped a version. Cloud platforms virtualized it. Containers packaged it again. IoT devices squeezed it into a few kilobytes of flash memory. None of those implementations exists today in the form it first shipped. The rules they were all built to satisfy do.

> **The specification outlived every machine built to satisfy it, including machines that did not exist when the specification was written.**

RFC 791 and RFC 793 persisted for a reason that goes beyond networking. They described a stable thing, what has to happen, and left an unstable thing, how any one system happens to build it, entirely open. Technology kept changing underneath them for exactly that reason, without ever touching what they said.

The same discipline, at the scale of one organization

This book takes that fifty-year-old lesson and applies it somewhere much smaller: a single team, a single codebase, a single organization's business rules. Most of those organizations still do the opposite of what the internet's protocol designers did. They let the source code carry the business knowledge. They lose that knowledge the moment the language or the platform changes. They rediscover it, expensively, whenever a system finally gets rewritten and someone has to work out what the old one actually did.

AI coding assistants made that problem visible almost overnight. Ask an assistant to build a feature from a short description, and it will produce something. If nobody wrote down the actual rule, the assistant invents a plausible one, filling the gap with what businesses of that kind usually do rather than what this one decided. The rest of this book calls that the difference between asking AI to guess and asking it to implement a decision someone already made.

The methodology here is Simon Martinelli's, and it continues a lineage that already runs deep: structured analysis, use cases, domain-driven design, Agile practice, architecture decision records. Each of those contributed a piece of the same idea, that a system's rules deserve a durable form separate from whatever code happens to implement them today. What changes now is the economics. When implementing a feature took weeks, the hours spent writing a precise specification first were easy to skip. When an AI assistant can implement that same feature in minutes, skipping the specification stops being a shortcut and starts being the most expensive step in the process.

The payoff doesn't show up on day one. It shows up the first time a rule changes and nobody has to hunt through old code to find where it lived. It shows up again when a new engineer reads the specification instead of reverse-engineering the system from its side effects. It shows up a third time when the underlying platform gets replaced and the business rules simply move to the new one, the way TCP/IP moved from mainframes to phones without anyone renegotiating what a packet is.

One claim, three kinds of reader

The chapters ahead are written for whoever is closest to the work: the engineer writing a use case for the first time this week, the architect trying to keep a dozen AI-assisted teams from drifting apart, the manager deciding what "done" should mean once implementation is no longer the slow part. Each of them will find a different amount of technical detail useful, and the book is arranged so each can take only what they need.

What ties their chapters together is the same claim Postel's generation made about the network, applied to a single company instead of the whole internet: write down what the system must do, keep that document separate from how any particular platform happens to build it, and the document will outlast the platform. RFC 791 did not know it would still be read in an era of phones, containers, and machine-written code. Your specification does not need to know what replaces your current stack either. It only has to say, precisely, what has to be true regardless.

Somewhere a copy of RFC 793 is still sitting in a standards archive, unglamorous and unchanged, outlasting every system ever built to satisfy it. This book is about writing your organization's version of that document, and about what changes, in how you work and what AI can do for you, once you have.

1

PART 1: FOUNDATION

Why AI needs a specification, not a prompt

Why does asking AI to build a feature from a short description produce code a team can't keep?

A developer at Riverside Clinics opens an AI assistant and types one line: "Build a feature that lets patients book appointments online." Ninety seconds later, she has working code. A booking form. A database table. Validation logic. A handful of passing tests. It looks finished.

She opens a pull request. Her team reads it and starts finding problems within the first few minutes. The booking form lets a patient pick any doctor at any clinic location, but Riverside's doctors only work specific locations on specific days, a rule everyone on the team knows and nobody wrote down. The appointment status field uses values the rest of the system doesn't recognize. The error handling doesn't match anything else in the codebase. None of this code is broken. It runs. It would even satisfy someone who had never seen Riverside's other systems. It is code built from a guess about how a clinic group probably works. Not for this one.

■ CHAPTER PROMISE

By the end of this chapter, you can tell a request that asks AI to guess your business apart from one that asks it to implement a decision your team has already made, and explain why that difference decides whether the code survives review.

Reading time: 11 minutes

The speed you gain, and where it goes

This scene repeats across organizations that adopt AI coding assistants expecting to move faster. Many of them do, for the first draft. Then a reviewer opens the pull request, and the hours saved writing the code get spent finding what it got wrong: the assumptions it made about location rules, status values, error conventions, none of them stated anywhere the AI could have read them.

The rewrite that follows takes hours of manual correction, sometimes days, done by the same developer the tool was supposed to speed up. Multiply that across a team shipping several features a week, and the productivity gain the tool promised doesn't show up in the sprint numbers. It shows up as review time, rework time, and a growing suspicion that the tool wasn't the upgrade it was sold as.

The tool isn't the problem. A traditional engineering team assumes the bottleneck is the person typing, and optimizes for that: faster typing, better frameworks, more automated tests. Much of the tooling built for developers has aimed at that assumption. AI coding assistants are the most extreme version of it, and they expose why the assumption was wrong. Developers were never the slow part. Figuring out what to build, whose rules applied, and how the new feature fit what already existed, that was always the slow part. It was just slow in a way that looked like typing, because writing the code was how a developer worked the problem out.

Take typing out of the loop entirely, hand it to a model that can produce a plausible booking form in ninety seconds, and the part that was never about typing is still there, waiting to be done by someone. The AI cannot do it for you, because it was never given the information that doing it requires.

Two different requests that sound almost the same

"Build a feature that lets patients book appointments" and "implement the appointment-booking use case" can look interchangeable. They are asking for two different kinds of work.

The first request describes an outcome and leaves every decision behind it unstated: which doctors are eligible, what counts as an available slot, what happens when two patients try to book the same slot at once, what the system calls a cancelled appointment versus a missed one. An AI assistant answering that request has to invent answers to all of it, because the request contains none of them. Most of the time, the invented answers are wrong. A guess about an organization it has never seen is usually a guess about the wrong organization.

The second kind of request names a decision that has already been made. Simon Martinelli, whose specification-driven methodology this book draws on throughout, puts the line there: the first request asks AI to reverse-engineer a business, the second asks it to translate a decision. A **specification**, the term this book uses for that decision written down, is a precise, technology-independent description of what a system must do: which actors are involved, what the rules are, what counts as success, and what counts as a failure the system has to handle. Writing one doesn't make the work disappear. It moves the work to where a human is still needed, deciding what the business actually requires, and away from where AI now does it faster than any developer can type: translating an already-made decision into working code.

> **Ask AI to guess your business, and it will. Ask it to implement a decision your team has already made, and it will do that instead.**

Telling the two kinds of request apart

Before asking an AI assistant to build anything, check which kind of request you're about to make. The test takes longer to explain than to run.

1. Write down the request exactly as you'd type it to the AI assistant, in full, before you send it.
2. Check it for three things: named actors (which users or roles, specifically), the data involved and any constraints on it, and at least one business rule that already exists somewhere, even if only in someone's head.
3. If the request names a feature but two or more of the three are missing, stop. Write down the missing decision first. A feature name is a label for the specification you haven't written yet.

Skip this test for a throwaway script or a small one-off change; neither needs it. Its job is to catch one moment: the point where a request about to go out is really a request for the AI to guess, before code comes back that nobody wanted.

The same feature, two different requests

Here is the Riverside Clinics request from the start of this chapter, run through the test, next to the version the rest of this book builds toward.

Guessing-shaped request

"Build a feature that lets patients book appointments online."

No actor named beyond "patients." No data constraints. No business rule. Every decision, which doctors, which locations, what counts as available, is left for the AI to invent.

Specification-shaped request

"Implement the Schedule Appointment use case (Chapter 3) against the Riverside Clinics domain model (Chapter 4): a patient books one open slot with one eligible doctor at one location, subject to the 90-day booking window agreed with clinic operations, following the error-handling and validation conventions in the Spring Boot skill (Chapter 7)."

Every decision the first version left open is either answered or points to where the answer already lives.

The rest of this book is spent building the documents that second request depends on: a stated reason the feature exists, requirements precise enough to test, a domain model of patients and doctors and appointments, and the architectural and implementation conventions that keep this feature consistent with everything else Riverside has built.

Where this test breaks down

▲ WARNING

The test in this chapter flags an obviously under-specified request. It does not guarantee a good specification. A request can name actors, data, and a rule, and still be wrong, if the underlying business decision was never actually settled, only assumed. Writing it down doesn't fix a decision nobody made correctly in the first place; it just makes the mistake visible sooner, and in front of the right people.

Treating every request as needing a full specification	A team spends a day documenting a five-line configuration change, and starts resenting the process instead of the code review it was meant to prevent.
Treating a specification-shaped request as automatically correct	A precisely worded request built on a wrong assumption produces precisely wrong code, faster than a vague one would have.

Field guide: the specification-shaped request test

Actors	Guessing-shaped: "patients," "customers," nobody in particular. Specification-shaped: which users, in which role, named as specifically as the business names them.
Data and constraints	Guessing-shaped: no constraint on anything. Specification-shaped: the data involved and the limit that applies to it, such as Riverside's 90-day booking window.
Existing business rule	Guessing-shaped: every rule left for the AI to invent. Specification-shaped: at least one rule that already exists somewhere, even if only agreed verbally.

If a marker is missing, there is one more question to ask: is there a person who can supply it before the request goes to AI, rather than after the code comes back?

Run this before sending a feature request to an AI assistant. If two or more of these come back on the guessing side, the request is asking AI to guess.

What to remember

- The bottleneck in software delivery was never typing speed. It was always understanding what to build; AI has only made that gap visible by removing the typing.
- A request that names only a feature asks AI to invent the business decisions behind it. A request that names actors, data, and rules asks AI to implement decisions a team already made.
- Code that runs correctly can still be wrong for a project, if it wasn't built from that project's own decisions.
- Writing a specification doesn't remove the hard part of the work. It moves it earlier, to the person who understands the business, and away from the part AI now does faster than any developer types.

QUESTIONS FOR YOUR TEAM

- The last time your team rejected AI-generated code in review, was it broken, or was it built on the wrong assumptions?
- How many of the feature requests your team gave an AI assistant this month would pass the test in this chapter?

ONE ACTION TO TAKE NEXT

Pull up the last feature request your team gave an AI assistant. Run it through this chapter's field guide before you ask the assistant for anything else.

TRANSITION. Most teams skip the first layer of a specification entirely: a stated reason the system exists. The next chapter is about why that costs more than writing it down would have.

Why AI needs a specification, not a prompt ends here.

2

PART 1: FOUNDATION

Why purpose comes before design

Why does skipping the vision step cost more than writing the vision would have?

The Riverside Clinics team has learned the lesson from the last chapter. Their next request to the AI assistant names actors, data, and a rule, not just a feature label. A product manager reads it over and likes the improvement. Then a colleague asks a question the request still doesn't answer: why are we building this, and for whom, exactly?

The product manager opens a blank document and writes a sentence in under five minutes. "Build a system that lets patients book appointments online instead of calling the clinic." It reads like an answer. It is the Chapter 1 request with a clause bolted on.

CHAPTER PROMISE

By the end of this chapter, you can write a vision statement that names the actual problem behind a feature request instead of the feature itself, and use it to surface a barrier your original request never mentioned, before a single line of code gets written around the wrong one.

Reading time: 11 minutes

A sentence that restates the request instead of explaining it

The product manager's sentence looks like a vision. It reads in prose instead of a ticket title, and it names patients and calling and booking online. But it answers a question nobody asked. It says what the system will do. It says nothing about why patients call today, or whether every patient who calls has the same reason for it. A team could write ten versions of that sentence, each better worded than the last, and learn nothing new about the people they are building for.

That gap costs more once it's found late than it would have cost to close early. Nobody asked why patients pick up the phone at all, rather than walking up to the front desk. So nobody could know there might be more than one reason, or that one of those reasons has nothing to do with convenience. By the time the booking flow is designed and built, finding a missed reason means reopening a decision that now has a database schema and a user interface built around it, not adding a paragraph to a document.

Scope decided late is scope decided by default. If nobody writes down what the system is and isn't responsible for, that boundary still gets set, just later, and by whatever the first architecture happens to support rather than by anyone's judgment. A vision written before any of that exists turns the boundary into a decision the team makes on purpose.

What a vision actually answers

A **vision** is a short, explicit statement of why a system exists and who it exists for: the problem it solves, the people who have that problem, and the boundary around what the system is responsible for and what it isn't. What the system does is a separate question, answered by the layers built on top of it, starting with the next chapter.

Vision and requirements get confused constantly, because both eventually sit in the same folder. They aren't interchangeable. A requirement is testable: the system must accept a booking request up to ninety days in advance. A vision explains why that requirement is worth having at all: patients are giving up on booking, and the clinic wants to know why before it decides how far out a slot should be offered. Requirements change often, as the team learns more about the solution it's building. Vision changes rarely, because the underlying problem rarely does.

> **A vision answers why. Everything written after it, requirements, use cases, a domain model, answers what and how. Get why wrong, and precision about what and how won't save the project.**

One question, asked at several sizes

Vision doesn't only apply to a single feature. Riverside Clinics, the organization, has a reason it exists: patients need care, and the clinics providing it need to run without every hour going to logistics instead of medicine. The patient portal, one product inside that organization, has a narrower reason: the phone is the slowest and least private way for a patient to reach the clinic, and the portal is meant to be the alternative. Online appointment booking, one feature inside that portal, has a narrower reason still, the one this chapter is building toward.

Each level answers the same question, why does this exist, at its own scope. The feature's vision has to fit inside the product's vision, and the product's vision has to fit inside the organization's. A booking feature that solves a problem the patient portal was never meant to solve is a sign that someone skipped a level, not that the feature is wrong on its own terms.

Finding the reason nobody wrote down

A vision that only restates the request usually comes from writing it alone, at a desk, from the ticket. The fix is a short conversation with the people closest to the current, broken way of doing things, run before the vision gets written rather than after.

1. Gather the people who deal with the current process daily, not only whoever requested the feature: front desk staff, a nurse or clinician, and a patient if one is reachable.
2. Ask three questions, in this order: what's wrong with how this works today, who feels that most directly, and why hasn't it already been fixed.
3. Listen for an answer that changes or adds to the original request, not one that only confirms it. A room that agrees with the ticket in the first two minutes hasn't been asked the right question yet.
4. Write the answer down as a sentence naming the specific problem and the specific people affected, not the feature that might solve it.

One short meeting with the right people in the room is usually enough. The cost is a meeting. The alternative is finding out after launch.

The Riverside booking vision, badly and then well

The product manager's sentence from the start of this chapter is the poorly written version below. Run through the conversation from the previous page, it doesn't survive intact.

The front desk lead is in the room, and so is a nurse who spends part of every shift on the phone. The nurse says the quiet part out loud: patients calling to book aren't only losing time on hold. Some patients never call, and never walk up to the counter either, because doing either one means stating the reason for the visit where staff, and sometimes other patients in the waiting room, can hear it, a mental health follow-up, or a test result they'd rather not announce out loud. Those patients don't complain about wait times. They just don't book, and the clinic never learns they needed care. That one sentence turns a vague convenience problem into a specific one.

Poor vision

```
"Build a system that lets patients book appointments online instead of calling the clinic."
```

Working vision

```
Patients currently book by phone or in person at the front desk. Which doctor is at which location on which day is knowledge only the front desk holds, so a patient booked at the wrong site arrives and is turned away. Both routes also require stating the reason for a visit where staff, and sometimes other patients, can hear it. Some patients tolerate that. Others avoid booking entirely rather than disclose it aloud, and the clinic never learns they needed care.

We want patients to book, reschedule, and cancel appointments online, privately, without narrating the reason for the visit to anyone. This is for patients who currently go without care rather than disclose it in person, and for front desk staff spending a large share of their day on calls a private alternative could remove.

This system is responsible for booking, availability, and confirmation. It is not responsible for billing, medical records, or clinical intake, which stay with the systems that already handle them. Success looks like fewer routine calls to the front desk, and bookings for visit types patients previously avoided scheduling in person.
```

Where a vision statement stops helping

▲ **WARNING**

A vision that names a problem is not proof the problem was diagnosed correctly. The stakeholder conversation in this chapter surfaces a barrier the original request missed; it doesn't confirm that the barrier it surfaces is the only one, or the biggest one. Writing an assumption down makes it visible. It doesn't make it true.

Treating vision as something written once and never revisited	Riverside's patient population, referral mix, and locations change over a few years; a vision frozen from the launch conversation keeps the team solving a version of the problem that no longer exists.
Writing vision by committee, aiming for agreement rather than clarity	A vision drafted to satisfy everyone in the room tends to soften back into the restated request this chapter opened with, because specificity is usually what disagreement gets negotiated away.

Field guide: the vision-document outline

- ☐ **The Problem.** What's unsatisfactory about how this works today, stated as a fact about the world, not a missing feature?
- ☐ **The Stakeholders.** Who has this problem directly, and who else is affected, including anyone who currently avoids the process rather than tolerates it?
- ☐ **The Scope.** What is this system responsible for, and what is explicitly left to something else?
- ☐ **The Approach.** In one or two sentences, what general approach addresses the problem, without describing the implementation?
- ☐ **The Success Criteria.** What observable change tells the team the problem is actually being solved?
- ☐ **Constraints.** What has already been decided, or must remain true, regardless of how the rest gets built?

Fill in one sentence per line before writing anything longer. A line you can't fill in is a stakeholder conversation you haven't had yet.

What to remember

- A vision answers why a system exists and for whom. What the system does belongs to the layers built on top of it.
- A vision that only restates the feature request has the shape of an answer without any of the content; it teaches the team nothing it didn't already know.
- The barrier a request never mentions usually surfaces in conversation with the people closest to the current process, not at a desk with the ticket alone.
- Vision fixes scope before architecture does it by default. Skipping it hands that decision to whatever gets built first.
- The same question, why does this exist, applies at the scale of a feature, a product, and an organization, each nested inside the one above it.

QUESTIONS FOR YOUR TEAM

- If you asked three people on your team why your current project exists, would they give the same answer, or three different ones?
- Is there a group of users your last feature request never mentioned, because nobody in the room had asked them anything?

ONE ACTION TO TAKE NEXT

Take the vision statement for whatever you're building now, or write one in five minutes if none exists. Read it against the field guide on the previous page. If it only restates the feature, find one person closest to the current process and ask the three questions from this chapter before you write the next draft.

TRANSITION. A vision names the problem and the people who have it. It doesn't yet say what the system must do about them, precisely enough for an AI assistant to build it correctly. The next chapter turns that vision into requirements.

Why purpose comes before design ends here.

3

PART 1: FOUNDATION

Specifying what the system must do

How do you turn a vision into requirements an AI assistant can't misread?

The Riverside Clinics vision from the last chapter settles why the system exists: some patients go without care rather than state the reason for a visit where staff and other patients can hear it, and the clinic wants booking to be private. Set against the one-line request that started this book, that is a large step forward. A developer picks it up to start building the booking flow, and inside the first hour she runs out of things the vision tells her.

What happens if a patient tries to book at three in the morning? The vision hasn't said. How far into the future can someone book, next week, next year, ever? It hasn't said that either. What happens when two patients grab the same ten o'clock slot half a second apart? Nothing in the vision answers that. The vision explained why the system exists. It never promised to explain what the system does once it's running.

CHAPTER PROMISE

By the end of this chapter, you can pin one piece of system behavior down precisely enough that an AI assistant tests it instead of guessing it, and turn a whole booking interaction into a single, step-by-step description complete enough to implement directly.

Reading time: 12 minutes

Where the unanswered questions go instead of into a document

Nobody wrote the three questions from the opening scene into a document, but someone still has to answer them before the booking form works. Left unstated, the AI answers them the moment it writes the reservation logic, and it answers with a plausible default instead of a decision. It might let patients book a year out, because nothing told it ninety days was the limit doctors' schedules support. It might let two patients reserve the same slot in the gap between checking availability and confirming it, because nothing told it that gap mattered enough to close.

The code runs. It passes a demo. None of that tells you whether the horizon it picked, or the race condition it left open, matches what Riverside's clinics can actually support.

A patient books an appointment fourteen months out. The clinic has no idea whether that doctor will still be practicing there by then. Nobody notices until the appointment is already confirmed and the patient has already taken the afternoon off work. That failure traces back to one line nobody wrote down: how far in advance is a booking allowed to reach.

Someone still has to answer questions like that before code does, and writing the answer down doesn't invent new work. It moves a decision that was always going to get made, out of the AI's guess and into something a human actually chose.

The unit AI implements best

A **requirement** is a statement about system behavior you can observe and test, and it says nothing about how the code achieves it. "The system should be user-friendly" fails that test twice over: nobody can watch the system and confirm it, and nobody can write a test case that passes or fails against a feeling. "A patient can complete a booking in three minutes or less, in five clicks or fewer" passes both. You can time it. You can count the clicks. You can tell, without asking anyone's opinion, whether the system meets it.

A single requirement describes one fact in isolation, though. "The system displays available appointment slots." "The system validates the patient's email address." "The system sends a confirmation within one minute." Each one is testable on its own, and none of them says how the facts fit together, or in what order, or what happens when one of them fails partway through. A patient's work rarely arrives as an isolated fact. It arrives as a trip from "I need an appointment" to "I have one," and that whole path is what a requirement list was never built to describe.

A **use case** describes that whole path: one actor, pursuing one goal, from start to finish, as a sequence of steps a system and a person carry out together. "Patient books an appointment" is a use case. It contains most of the requirements above, in the order they actually happen, plus the moments where something can go wrong along the way.

> **A requirement tells an AI assistant one fact about the system. A use case tells it the whole interaction that fact belongs to. Handing over the whole interaction is the core of the methodology this book draws on throughout, Simon Martinelli's: an assistant that can see how the steps relate has less left to assemble on its own.**

Turning an open question into a written rule

Requirements rarely arrive fully formed. They surface in conversation, usually between someone who understands the users, someone who understands the business constraints, and someone who understands the rule that's been sitting in a colleague's head for years without ever reaching a document. Here is that kind of conversation, condensed, negotiating the booking horizon the opening scene left open.

A workshop, condensed

Facilitator: What has to be true before a patient can book?

Domain expert: They need to see when the doctor is free.

Product owner: Only during the hours we're open. We close at five.

Business analyst: We actually want booking available around the clock, any day, for future dates. Just not too far out.

Domain expert: How far is too far?

Business analyst: Doctors' schedules are only confirmed ninety days ahead. Past that, nobody knows who's working.

Facilitator: So the rule is: patients can book online any time, any day, but never more than ninety days before the appointment.

Illustrative, built to show how a workshop narrows a vague rule to a specific number. The shape of the conversation is typical of how this kind of decision gets made; the words are not a transcript of an actual meeting.

1. Ask what has to be true before the action happens, and write down the first answer, even when it's vague.
2. Push on every edge you can think of: after hours, a year from now, two people acting at the same instant. Most vague answers survive the first push and fail the second.
3. Stop when someone names an actual number or limit. Vague phrases like "not too far out" are a sign to keep pushing until a specific figure, like "ninety days," replaces them.
4. Write the rule as one sentence a developer, or an AI assistant, could test against, and keep it where both can find it.

The Schedule Appointment use case, start to finish

Filled in below, including the ninety-day rule the workshop just settled.

Use case: Schedule Appointment

Actor: Clinic patient, or a receptionist booking on the patient's behalf.

Preconditions:

- Patient, or a receptionist on the patient's behalf, is registered and logged in.
- A doctor has open availability within clinic hours (8 a.m.–5 p.m., Mon–Fri).

Main success scenario:

1. Patient selects a doctor.
2. System displays that doctor's open slots, fifteen-minute intervals.
3. Patient selects a time.
4. System re-checks that the slot is open, then reserves it.
5. Patient confirms the booking.
6. System emails confirmation: doctor, time, location, cancellation link.
7. System shows a booking summary; the appointment appears on the patient's dashboard and the doctor's schedule.

Alternative flows:

- **No slots available, step 2:** offer a different doctor or a later date range.
- **Slot taken before reservation, step 4:** notify the patient, discard the attempt, redisplay availability.
- **Patient cancels, any step:** discard the reservation, return to doctor selection.

Postconditions:

- Appointment exists, linked to patient and doctor; confirmation email sent.
- Doctor's schedule and patient's dashboard both reflect the appointment.

Business rules:

- No booking more than 90 days ahead, the confirmed horizon for doctors' schedules; slots run fifteen minutes, within clinic hours (8 a.m.–5 p.m., Mon–Fri).
- Cancellation requires at least 24 hours' notice; otherwise, call the clinic.
- Doctor availability is entered by clinic staff, not pulled from an external calendar.

Where a use case can look finished and still mislead

▲ WARNING

A use case can name every step and still leave the work half done. If a step says "the system sends a confirmation email" and never says what the email contains, the AI assistant has to invent a subject line, a greeting, and a layout, and it will. If one use case calls the actor "Doctor" and another calls the same person "Physician," the AI assistant has no way to know they're the same thing. Before any use case goes to an AI assistant, check that every step is filled in and that its terms match every other use case in the specification.

Handing an AI assistant a backlog of user stories instead of a use case	"As a patient, I want to book an appointment online, so I don't have to call" plans the work well and specifies almost nothing: it doesn't say what happens with no slots free, or two patients racing for one. A single story like this one usually bundles several use cases together, and an AI assistant asked to implement it has to split that bundling apart itself.
Writing use cases for a roadmap conversation	A use case's preconditions and alternative flows are exactly the detail a planning meeting doesn't need and a backlog groom shouldn't have to sit through. User stories exist for that conversation. Use cases exist for the one an AI assistant needs before it writes anything.

Field guide: the use-case template

This is the shape Martinelli's methodology recommends: actor, preconditions, main scenario, alternative flows, postconditions, business rules. The template below is that shape with the blanks named.

- ☐ Title: one goal, usually a verb phrase, one actor accomplishing it.
- ☐ Actor: who performs this use case, named specifically, not "the user."
- ☐ Preconditions: everything that must already be true before step one can run.
- ☐ Main success scenario: the complete happy path, numbered, one action per step.
- ☐ Alternative flows: every place the main scenario can go wrong, each tied to the step where it branches.
- ☐ Postconditions: everything that must be true once the use case has succeeded.
- ☐ Business rules: the constraints that apply across the whole use case, each written as a testable limit.

Fill in every line before a use case goes to an AI assistant. A blank line here is a decision the AI assistant will make for you.

What to remember

- A requirement is testable, observable, and silent on implementation. A sentence nobody can test is only an opinion.
- A vision leaves decisions open, like booking horizons and race conditions between two people acting at once. A workshop that pushes past the first answer is what turns an open decision into a written rule.
- A use case describes one complete interaction rather than an isolated fact, and that completeness is why this book treats a use case, not a requirement list, as the unit to hand an AI assistant.
- User stories plan work; use cases drive implementation. Use one where the other belongs and you get a backlog nobody can build from, or a specification nobody can prioritize against.

QUESTIONS FOR YOUR TEAM

- How many requirements has your team written this quarter that name an actual number, against how many only describe a feeling, like "fast" or "easy"?
- If you handed an AI assistant your last three user stories with nothing else, what would it have had to guess?

ONE ACTION TO TAKE NEXT

Take the next use case your team plans to hand to an AI assistant and run it against this chapter's field guide before it goes anywhere. Count the lines you cannot fill in; each one is a question for someone outside your team.

TRANSITION. Every step in the Schedule Appointment use case talks about patients, doctors, and appointments as though their shape is already agreed. The next chapter settles that agreement: a domain model precise enough that an AI assistant stops inventing what a doctor record contains.

Specifying what the system must do ends here.

4

PART 1: FOUNDATION

Modeling the business objects a system runs on

What must AI know about your business objects before it can implement anything correctly?

At Riverside Clinics, a developer picks up the Schedule Appointment use case from Chapter 3 and hands it to an AI assistant with one instruction: implement this. The use case names the actor, the preconditions, the seven-step booking flow, the ninety-day window clinic operations agreed to. It reads like exactly the specification-shaped request Chapter 1 asked for. The code that comes back compiles, passes its own tests, and looks finished. Nobody catches what it got wrong until a patient shows up at the wrong site: the code assumes every doctor works exactly one clinic, because nothing in the use case said otherwise, and half of Riverside's doctors rotate between two.

CHAPTER PROMISE

By the end of this chapter, you can build a domain model, entities, value objects, enumerations, and the relationships between them, precise enough that an AI assistant stops inventing your business objects and starts implementing the ones you already have.

Reading time: 10 minutes

What AI invents when nobody names the objects

A use case tells AI what happens: who does what, in what order, under what conditions. It does not tell AI what the actors and the things they act on actually are. The Schedule Appointment use case says a patient books an open slot with an eligible doctor. It never says whether a doctor can work more than one clinic, what values an appointment's status can take, or whether a patient record and a medical record are the same object or two objects that travel together. Somebody has to answer these questions before the code is correct. If nobody does, AI answers them anyway.

Its answers are guesses shaped like decisions. Riverside's AI assistant guessed a one-to-one relationship between doctor and clinic, because the use case never mentioned two, and one is the simpler assumption to build against. It guessed appointment statuses of "Booked" and "Done," because those are ordinary values in appointment systems generally, not because anyone at Riverside uses those words. The front desk staff who use Riverside's existing systems call them "Scheduled" and "Completed." Neither guess broke a test. Both guesses were wrong.

The cost shows up downstream, well after the pull request closes. A report built against "Booked" returns zero appointments, because nothing in the database matches that value. A doctor who splits her week between two clinics gets scheduled as if she only worked one, and a patient books a slot the doctor isn't actually staffing that day. The tests were never going to catch this. It is a business fact nobody told the AI, arriving in the finished system as if the AI had decided it, because it had.

What a domain model actually describes

A **domain model** describes the business objects a system deals with, independent of how any database happens to store them: what things exist, what facts are true about each one, and how they relate to each other. It stays the same regardless of which entity-relationship diagram a particular database uses, or which class hierarchy a particular language happens to prefer. Riverside's domain model says the same thing whether the system behind it runs on Postgres or MongoDB, Java or Python, because it describes the clinic's business rather than any one technology's way of representing it.

The model is built from three different kinds of business fact, and mixing them up is a common way domain models go wrong. An **entity** is something the business tracks as itself over time, something with its own identity that persists even as its attributes change: a Patient stays the same patient after a phone number changes, a Doctor stays the same doctor after moving offices. A **value object** is a fact fully described by its own value, with no identity of its own to track, and replaced rather than edited when it changes: two appointments scheduled for the same fifteen-minute window on the same day describe the identical time slot, the same fact recorded twice rather than two different facts that happen to look alike. An **enumeration** is an attribute limited to a fixed, named set of valid values: an appointment's status is Scheduled, Completed, Cancelled, or NoShow, never a fifth value someone typed by hand.

None of this is new. The distinction between entities and value objects, and the discipline of naming a domain model before writing code, comes from established software design practice, most visibly the domain-driven design tradition Eric Evans described. Riverside doesn't need to reinvent it. It needs to apply it once, clearly, to patients, doctors, and appointments.

> **A domain model names what your business already believes to be true. It doesn't invent anything, and neither should the AI reading it.**

Building the model in four passes

1. List every noun your use cases already depend on: patient, doctor, appointment, clinic, time slot, without deciding yet what kind of fact each one is.
2. Classify what you listed. For each noun, ask whether it needs an identity you'll refer back to later (an entity), whether it's fully described by its current value with no identity of its own (a value object), or whether it's really just an attribute limited to a fixed set of options (an enumeration).
3. For each entity and value object, name its attributes concretely, not "details" but "phone_number," and name its relationships to other entities, including the cardinality: one, many, or exactly one on each side.
4. Check the model against your use cases in both directions. Every noun a use case depends on should appear in the model. Every entity in the model should be used by at least one use case; if one isn't, remove it.

The Riverside Clinics domain model, named once

Run patients, doctors, appointments, clinics, and time slots through the four passes, and here is what the model looks like for the slice of Riverside's business the Schedule Appointment use case touches. This is an excerpt. Riverside's real model also covers billing and insurance. It's complete enough, though, that an AI assistant implementing the booking flow no longer has to guess any of it.

Riverside Clinics domain model (excerpt)

- **Patient**: a person who receives care. Attributes: medical_record_number (unique), date_of_birth. Relationship: has many Appointments.
- **Doctor**: a clinician who provides care. Attributes: specialty (enumeration: Cardiology, Dermatology, Orthopedics, Family Medicine, Psychiatry), years_licensed. Relationship: works at one or more Clinics (many-to-many).
- **Appointment**: a scheduled visit. Attributes: scheduled_slot (TimeSlot value object), status (enumeration: Scheduled, Completed, Cancelled, NoShow). Relationship: involves exactly one Patient and exactly one Doctor, at exactly one Clinic.
- **Clinic**: one physical Riverside location. Attributes: name, business_hours. Relationship: employs many Doctors.
- **TimeSlot** (value object): a fifteen-minute window with no identity of its own. Attributes: date, start_time, end_time. Meaningful only as the scheduled_slot of an Appointment.

Compare this to what the AI assistant guessed at the start of this chapter. The relationship between Doctor and Clinic is many-to-many here, because the model says so in writing instead of leaving the AI to assume the simpler case. The status values are Scheduled, Completed, Cancelled, and NoShow, in the front desk's own words, the words the people who actually run the clinics use every day.

Where a domain model still goes wrong

▲ WARNING

A domain model can quietly turn into a database diagram. Riverside's real database splits a patient's address into its own table, to avoid storing the same address twice for a household sharing one home. The domain model still says, correctly, that a patient has one address. How the database avoids duplicating it is a storage decision, made later, by different people, for different reasons.

Modeling the database instead of the business	The model inherits the schema's normalization, foreign keys as relationships, join tables as entities of their own, and an AI implementing the next feature copies structure that was never a business decision in the first place.
Freezing the model after the first draft	Riverside adds telehealth visits eighteen months later, and Scheduled, Completed, Cancelled, and NoShow no longer cover what actually happens. A video visit that never connects sits between NoShow and Cancelled, and nobody updated the enumeration to say which one it is.

Field guide: telling an entity from a value object

- [ ] Does it need an identity you'd refer back to later, even after every one of its attributes changes? If so, it's an entity.
- [ ] Do two instances with identical attributes mean the same thing, or two different things that happen to look alike? The same thing means a value object; two different things mean an entity.
- [ ] Does it ever exist on its own, apart from the entity it describes? If it only ever appears as part of something else, it's a value object.
- [ ] When one of its parts changes, do you edit it in place or record a new one? A value object gets replaced whole; an entity gets edited and stays the same entity.
- [ ] Is it really just an attribute limited to a small, fixed, named set of options? Then it's an enumeration, neither an entity nor a value object.

Run this checklist noun by noun while you're building the model, before the code comes back mismatched.

What to remember

- Without a domain model, AI has to guess your entities, attributes, and relationships, and most of what it guesses is wrong for your business specifically, however ordinary each guess looks on its own.
- A domain model names business meaning; it is a different document from a database schema, and the two are allowed to disagree.
- Entities carry identity across changes, value objects are fully described by their own value, and enumerations limit an attribute to a fixed set of options. The model earns its keep once every business object is sorted correctly among the three.
- The model changes as the business changes. A Riverside that starts offering telehealth visits needs its Appointment status enumeration revisited then, on the same timeline as the change itself.

QUESTIONS FOR YOUR TEAM

- Which of your system's core objects exist only in a database migration file, invisible to any document a new hire could read in an afternoon?
- The last time your business added a new kind of record, telehealth, a new appointment status, a new product line, did anyone update the domain model, or only the code?

ONE ACTION TO TAKE NEXT

Pick the one entity in your system most likely to be modeled wrong right now, the one everyone assumes they already understand. Run it through this chapter's checklist before you trust an AI assistant with it again.

TRANSITION. Chapter 3 gave the system a use case. This chapter gave it a domain model. The next chapter puts a vision, requirements, use cases, and a domain model into one document, and follows the Riverside Clinics booking feature through all four layers at once.

Modeling the business objects a system runs on ends here.

5

PART 1: FOUNDATION

Bringing the specification together

How do the first four layers combine into a specification AI can implement directly?

Four documents exist for the Riverside Clinics booking feature by the time this chapter opens, and each one is good. The vision, written with the front desk lead and a nurse in the room, names the patients who never book at all rather than say the reason out loud. The requirement from the workshop pins the booking horizon down to ninety days, a number a test can check. The Schedule Appointment use case runs the whole interaction start to finish, alternative flows included. The domain model gives Patient, Doctor, Clinic, and Appointment their attributes and the rules connecting them. Written separately, reviewed separately, each one earned its place in the last three chapters.

CHAPTER PROMISE

By the end of this chapter, you can assemble a vision, a requirement, a use case, and a domain model into one specification document, cross-checked so the same term means the same thing in all four places, ready for an AI assistant to implement without guessing at the connections between them.

Reading time: 12 minutes

The cost of leaving four documents apart

They also live in four different places. The vision is a paragraph from a stakeholder meeting, still sitting in someone's notes. The requirement is a line item in a tracker. The use case is a document a business analyst wrote and a developer skimmed. The domain model is a diagram an architect drew on a whiteboard and photographed. When a developer sits down to implement the feature, she opens the AI assistant and pastes in the use case, because that's the document describing what the code has to do. She doesn't attach the domain model; it lives in a different tool. She doesn't attach the ninety-day requirement either; she has seen it before and assumes the use case already covers it. On that one she is right, because the use case carries the ninety-day rule in its business rules.

The code that comes back handles the happy path correctly and misses the fact that made this feature worth building in the first place: a Riverside doctor works at more than one clinic, so a booking has to name a location that doctor actually works. Nothing in the use case she pasted said that. It lived one document over, in the domain model she never attached.

The mistake in the last three chapters was missing information: no stated reason for the system, no requirement precise enough to test, no model of what a doctor and a patient are to the business. This chapter's mistake is different. Every piece of information exists. It simply isn't in the same room.

An AI assistant, like a new hire, implements from what it is handed. Hand it a complete use case and nothing else, and it will implement that use case correctly, on its own terms. It has no way of knowing that a business rule belonging to the use case lives in a domain model two documents away, because from where it sits, that document does not exist. The resulting code follows the instructions it was given, to the letter. Those instructions were incomplete because nobody had put the specification back together before handing it over, even though every piece of it already existed.

The review that follows looks identical to the one in Chapter 1: a developer finds the gap, traces it back, and either patches the code or goes looking for the missing piece. The hours lost are the same hours. The cause is different enough that most teams never connect the two problems, because the second one hides behind documents that all, individually, look finished.

One document, four questions answered

Each layer built in Chapters 2 through 4 answers a question the others don't. The vision answers why the system exists and for whom. The requirement answers what specific, observable behavior it must have. The use case answers what a complete interaction looks like, start to finish. The domain model answers what business objects the system runs on, and how they relate. None of the four can stand in for another. A vision that lists database fields has stopped being a vision. A domain model that describes user steps has stopped being a domain model.

A complete specification places its four documents in agreement with each other, in one place, before anyone, human or AI, starts implementing from any single one of them. Agreement here means something specific: the same term names the same thing everywhere it appears. If the vision names a rule about doctors and locations, that rule has to survive into the requirement as something testable, into the use case as a step or a business rule, and into the domain model as a relationship between two entities. A rule that appears in only one of the four layers is a rule your AI assistant may never see, because what it reads is what it was handed.

> **A specification is complete when its four layers agree with each other in one place, not when each layer merely exists.**

Assembling the four layers into one document

1. Place the vision at the top, in the words the stakeholders agreed to, not restated by whoever is assembling the document. A reader should meet the reason before the rule.
2. Attach only the requirements this feature's use case depends on, not the full backlog. A ninety-day booking window belongs here. An unrelated requirement about billing does not.
3. Add the use case in full: actor, preconditions, main scenario, alternative flows, postconditions, business rules. This is the shape an AI assistant implements best, and summarizing it defeats the reason for including it.
4. Add the domain model, limited to the entities and attributes the use case names. Then check each one against the use case by name; a use case that says "Doctor" and a model that says "Physician" are describing the same thing to a person and two different things to an AI assistant.
5. Version the whole assembly as one document, dated. When a requirement changes, this is the one file that changes, not four scattered guesses reconciled after the fact.

The Riverside Clinics specification, assembled

Here is what steps one and two produce for Riverside Clinics: the vision from Chapter 2, condensed, with the multi-location fact from Chapter 4's model added to it, followed by the one requirement this use case depends on most directly, the rule that turns "available" into something a system can check rather than something a receptionist remembers.

> **Specification: Schedule Appointment, Riverside Clinics (v1.0)**
>
> 1. VISION
>
> Patients book by phone or in person at the front desk. Which doctor is at which location on which day is knowledge only the front desk holds, so a patient booked at the wrong site arrives and is turned away. Both routes also require stating the reason for a visit where staff, and sometimes other patients, can hear it. Some patients tolerate that. Others avoid booking entirely rather than disclose it aloud, and the clinic never learns they needed care. We want patients to book, reschedule, and cancel appointments online, privately. This system is not responsible for billing, medical records, or clinical intake, which stay with the systems that already handle them. Chapter 4's model then recorded in writing what the vision had left as front desk knowledge: a doctor works at one or more clinics.
>
> 2. REQUIREMENT (REQ-07: availability)
>
> A time slot is available for booking only if all three hold: it falls within 90 days of the current date; the doctor works at that location; and no existing appointment already occupies it. The system must not display a slot to a patient unless all three are true, and must state which condition failed when a patient requests one that isn't.

The specification, continued

The use case below is the one Chapter 3 wrote, condensed. In the working document it sits at full length, exactly as Chapter 3 has it, every step and every alternative flow written out.

Specification: Schedule Appointment, Riverside Clinics (continued)

```
3. USE CASE: Schedule Appointment

Actor: registered patient, or a receptionist booking on the patient's behalf.
Preconditions: patient is logged in; at least one doctor is accepting bookings
within the request window.

Main scenario: (1) patient selects a location or a doctor; (2) system displays
only slots satisfying REQ-07; (3) patient selects a slot; (4) system reserves
the slot and creates the appointment record; (5) system sends confirmation
naming the doctor, date, time, and location.

Alternative flows: if no slot satisfies all three conditions, the system
offers the same doctor at a later date or another doctor with availability in
the requested range, rather than a bare "not available." If a slot is taken
between steps 2 and 4, the system notifies the patient and redisplays current
availability.

Postconditions: the appointment exists, tied to one patient, one doctor, one
location, one time slot; confirmation has been sent.

Business rules: slots run fifteen minutes, inside clinic hours; cancellation
requires at least 24 hours' notice, otherwise the patient calls the clinic;
doctor availability is entered by clinic staff, not pulled from an external
calendar.
```

The specification, concluded

Specification: Schedule Appointment, Riverside Clinics (concluded)

```
4. DOMAIN MODEL (entities the use case above names)

Patient: a registered account; has many Appointments. Doctor: works at one or
more Clinics, many-to-many, the relationship this whole feature turns on; has
many Appointments. Clinic: one physical Riverside location; employs many
Doctors, handles many Appointments. Appointment: belongs to one Patient, one
Doctor, one Clinic, one TimeSlot; status is Scheduled, Completed, Cancelled,
or NoShow. TimeSlot (value object): date, start time, end time; has no
identity of its own outside an Appointment.
```

Read start to finish, one fact threads through all four sections in the same words. What the vision records as a doctor working at one or more clinics becomes the second condition in REQ-07, becomes the filter the use case applies before it shows a patient anything, and becomes the many-to-many Doctor-Clinic relationship in the domain model. An AI assistant implementing from this one document doesn't have to go looking for that connection. It has already been made.

Where assembly still goes wrong

▲ **WARNING**

Assembling the four layers once is not the same as keeping them assembled. A specification that stops changing the day the feature ships decays the same way any other documentation decays, unnoticed until it no longer describes what is running.

Treating assembly as a one-time step	Six months after launch, a rule changes in production and nobody updates the assembled document. New work against the feature reverts to guessing, because the one place that was supposed to hold the truth no longer does.
Assembling everything instead of what one use case needs	Pulling in the full domain model and the entire requirements backlog produces a document long enough that reviewers stop reading it, which defeats the reason it was assembled in the first place.

Field guide: the one-page assembly template

Six sections, one file. The note against each says what belongs there, and nothing about how to word it.

- ☐ Header: feature name, version number, date. One line, so a reader knows which assembly they are holding and which one it replaced.
- ☐ 1. Vision: why this exists and for whom, in the words the stakeholders agreed to.
- ☐ 2. Requirements: only the ones this use case depends on, each written as something a test can check.
- ☐ 3. Use case: actor, preconditions, main scenario, alternative flows, postconditions, business rules. All six lines filled.
- ☐ 4. Domain model: the entities, attributes, and relationships the use case names, under the names the use case uses.
- ☐ 5. Sign-off: the date the four sections above were last cross-checked against each other, so a reader knows how current the agreement between them is.

Fill every section before handing the document to an AI assistant or a new team member. A section left blank is a decision someone downstream makes on your behalf, usually without knowing they made it.

What to remember

- A specification is complete when its four layers exist together and agree with each other, not when each one merely exists somewhere.
- Assembling costs time up front. That cost is repaid the first time a requirement changes, since one document gets updated instead of four that nobody reconciles.
- The assembled specification, not the code generated from it, is what should still be correct after the code has been rewritten twice.
- Consistency between layers, the same term meaning the same thing everywhere it appears, matters more than how complete any single layer is on its own.

QUESTIONS FOR YOUR TEAM

- Can anyone here point to one document containing this feature's vision, requirement, use case, and domain model together, or does it take four open tabs?
- The last time a requirement changed, did the domain model and the use case change with it, or only the code?

ONE ACTION TO TAKE NEXT

Take a feature currently in flight. Gather whatever exists for it, vision, requirements, use case, domain model, even if some of it is incomplete, and assemble it into one document today. Cross-check it once against the field guide on the previous page before anyone implements another line against it.

TRANSITION. This assembled document tells an AI assistant what to build and why. It doesn't yet tell it how: which architecture, which layering, which naming rules apply to code that has to sit next to everything else Riverside has already built. Architecture is the layer the next chapter adds. The two chapters after it add the reusable implementation knowledge a team encodes once, and the workflow that carries a finished specification through to shipped code.

Bringing the specification together ends here.

6

PART 2: FRAMEWORK

Architecture as context

How do structural constraints keep AI-generated code consistent with everything around it?

The Riverside Clinics booking specification from the last chapter is finished, and three developers pick it up the same afternoon. One takes patient lookup. One takes slot availability. One takes doctor eligibility. Each opens an AI assistant, describes a slice of the use case, and gets back working code in minutes: methods that compile, pass their own tests, and do exactly what was asked. All three branches are ready to merge into the same feature by end of day.

CHAPTER PROMISE

By the end of this chapter, you can name the layering, dependency, and naming rules your own codebase already follows without anyone having written them down, and turn them into a short document AI reads before it writes a line, instead of guessing at them one request at a time.

Reading time: 12 minutes

Three correct answers that don't agree with each other

Open the three branches side by side and each one is defensible on its own. The patient-lookup developer's assistant produced a `PatientLookupService` that calls a repository, which calls the database, which is exactly how the rest of Riverside's system is built. The slot-availability developer's assistant produced a `DoctorAvailabilityManager` that queries the database directly from what should have been the application layer, skipping the repository entirely because nothing told it a repository was expected there. The doctor-eligibility developer's assistant, asked to check whether a doctor could take a given slot, decided the cleanest design was to publish an internal event once eligibility was confirmed, an event nothing else in the system subscribes to or ever will.

Each of the three passed its own unit tests. None of the three is wrong in isolation. What they share is that nobody told any of the three assistants what kind of object this is supposed to be, so each one reached for a different, individually reasonable answer: a manager here, a service there, an event publisher somewhere a plain return value would have done the job. A reviewer looking at the merged branch is looking at three conventions where the project needs one, and a layer boundary one of the three breaks through.

This is the cost that doesn't show up until the second or third feature. The first divergence looks like a style nitpick, worth a comment and a quick fix. By the tenth feature, with no rule ever written down, the project's real architecture is whichever pattern happened to get written most often, and a new engineer reading the code has no way to tell a decision from an accident. Every mature codebase already runs on rules like these: controllers never call a repository directly, only the domain layer is allowed to raise a business event, validation happens in the application layer and nowhere else. Developers absorb rules like that by reading months of other people's pull requests. An AI assistant sees only the request in front of it. It has no pull requests to have read.

What architecture means here

Call this what it is: **architecture**. Not the boxes-and-arrows diagram a slide deck shows on day one, useful as that diagram can be. Architecture, in the sense this book uses the word, is the set of decisions that apply to every feature a team builds rather than just the one in front of it, and that change far more slowly than requirements do: which layer is allowed to call which, what a repository gets named, where a business rule is allowed to live, how a module's boundary gets drawn. A diagram unsupported by rules like these is illustration. A single sentence that states one of these rules, with no box drawn at all, is architecture.

Every one of Riverside's three implementations obeyed a perfectly valid, textbook pattern. None had been told which pattern this project uses, because nobody had written it down anywhere an AI assistant could read it. That gap is the same one running under this entire chapter, and it points at a change bigger than one codebase. For a few years, getting good output from an AI assistant meant refining the prompt: finding the right wording, trying it, retrying it, until the result looked acceptable. Architecture reframes that problem. Instead of wording each request more carefully, a team writes its structural rules once, in a document, and every future request gets shorter, because the rules no longer need repeating.

That change has a name: **context**, the written information an AI assistant is given about a project before it is asked to do anything, and context engineering, the discipline of building that information up once so it doesn't need re-explaining request by request. Architecture is the first piece of context in this book that scales across an entire codebase instead of one feature at a time.

> **Architecture settles the structural questions before AI ever reaches the business problem: which layer, which name, which boundary. What's left is the business decision itself, and the rest of the specification has already answered that one.**

Writing down the rules your codebase already has

Most teams already have an architecture; it just lives in habit rather than in writing. The work here is finding the rules already in force and putting them somewhere an AI assistant can read them before it starts guessing.

1. Name the layers your system actually has, and give each one exactly one responsibility. Three or four layers is typical for a system Riverside's size; more than that usually means two layers are doing the same job under different names.
2. State the dependency rule for those layers in plain sentences: which layer may depend on which, and which layer, usually the one holding business rules, may depend on nothing else in the system at all.
3. Fix one naming convention per kind of artifact: application service, repository, event, command. Write it down once rather than letting each developer's assistant choose its own.
4. Record the decisions that don't fit neatly into layers or names: error handling, security, testing strategy, deployment. Use a documentation format that already exists rather than inventing one: an arc42 template, a widely used structure for architecture documentation; architecture decision records (ADRs), short files that each capture one decision and the reason behind it; or C4 diagrams, a set of architecture views at increasing levels of zoom, from the whole system down to individual components. Simon Martinelli, whose specification-driven development methodology this book draws on throughout, recommends arc42. The format is widely known and already built to hold exactly this content. What matters is that the decisions end up somewhere durable, rather than in one developer's head.

Riverside Clinics: the rules made explicit

Riverside has been building software this way for years without ever writing the rules down. Here is what the team produces the week after the three-branch merge above, the first time anyone puts the project's actual layering and naming in writing.

Riverside Clinics: layers and the dependency rule

Presentation: handles requests from patients, staff, and the clinic's booking website
Application: coordinates one use case at a time, such as Schedule Appointment
Domain: the business rules for patients, doctors, appointments, and slots
Infrastructure: persistence, the scheduling database, outbound notifications

- `Presentation may depend on Application. Nothing may depend on Presentation.`
- `Application may depend on Domain. Nothing outside Application may call Infrastructure directly.`
- `Infrastructure may depend on Domain, to read and store domain objects.`
- `Domain depends on nothing else in the system. It is the one layer every other layer is written around.`

Riverside Clinics: naming, and the scenario run again

> **Riverside Clinics: naming, fixed once per kind of object**
>
> - An application service ends in Service: PatientService, not PatientManager, not PatientProcessor.
> - A repository ends in Repository: DoctorAvailabilityRepository.
> - A domain event ends in Event: AppointmentScheduledEvent, raised only from the domain layer.
> - A use-case command is named for the action it performs: BookAppointmentCommand, CancelAppointmentCommand.

Run the three-developer scenario again with this document sitting in the AI assistant's context, and the result changes shape. Patient lookup, slot availability, and doctor eligibility all become one kind of object, an application service depending on the domain, backed by a repository in infrastructure. Eligibility becomes a plain answer from a domain rule instead of an event nobody subscribes to. Three names for the same idea become one, and the layer boundary that broke on the first pass has nowhere left to break through.

Where written rules stop working

▲ WARNING

A document is not enforcement. Rules on a page decay the same way tribal knowledge did, if nobody checks the code against them; a repository that starts calling another repository directly will still pass every test that isn't looking for it. Pair the document with something automated, a dependency-direction linter or an architecture test, that fails a build the moment the written rule and the actual code stop agreeing.

Writing rules too rigid for the system's actual size	A four-person team adopts a layering scheme built for an organization many times its size because a template recommended it, and every trivial change now requires touching five files to satisfy a rule the project never needed.
Treating architecture as a substitute for the domain model	The layers are correct, the naming is consistent, and the business rule inside the domain layer is still wrong, because architecture governs where a decision lives, not whether the decision itself was made correctly.

Field guide: an architecture document scaled to one team

- ☐ Layers named, each with exactly one stated responsibility.
- ☐ The dependency rule written as plain sentences: which layer may call which, and which layer depends on nothing.
- ☐ A naming convention fixed per kind of artifact: service, repository, event, command.
- ☐ A module-boundary rule: organized by business capability, patients, appointments, billing, rather than by technical role.
- ☐ Error-handling, logging, and security conventions stated once, not left to whichever pattern the last pull request happened to use.
- ☐ Testing expectations per layer: what needs a unit test, what needs an integration test, what doesn't need either.
- ☐ A short decision log, one entry per architectural choice and its reason, for anything that doesn't fit cleanly into the sections above.

This is deliberately smaller than a full arc42 template or a complete C4 diagram set. Start with the sections above; grow into arc42, ADRs, or C4 as the team and the decisions worth recording both grow.

What to remember

- Code that is entirely correct can still be wrong for a project, if nobody told the AI assistant which structural rules this project follows.
- Every mature codebase already runs on invisible rules; developers absorb them by reading each other's work over months, and an AI assistant has no equivalent history to learn from.
- Architecture is the set of decisions that apply to every feature a team builds: layering, dependency direction, naming, module boundaries. A diagram only illustrates those decisions; it isn't one of them.
- The dependency rule is the one sentence that does the most work: state which layer depends on nothing, and most accidental coupling stops before it starts.
- Writing architecture down once is what turns prompt engineering, wording each request carefully, into context engineering, giving the AI the rules once so no future request has to restate them.

QUESTIONS FOR YOUR TEAM

- If three developers each asked an AI assistant to add a service class this week, would the three results share one naming convention, or three?
- Where does your team's dependency rule currently live: written down, or in the memory of whoever has been here longest?

ONE ACTION TO TAKE NEXT

Pick one system you maintain and write down its layers and its dependency rule in five sentences or fewer. If you can't finish in five sentences, that's the rule your next AI-generated pull request is about to violate.

TRANSITION. Architecture answers how this system is structured. It doesn't answer how this team writes a test, logs an error, or structures a service in this particular stack, decisions that differ from one organization to the next even under the same architecture. The next chapter turns that layer of convention into something reusable too.

Architecture as context ends here.

7

PART 2: FRAMEWORK

Skills: reusable implementation knowledge

How do you encode a team's implementation conventions once instead of repeating them in every prompt?

A developer at Riverside Clinics sits down to ask an AI assistant to implement the Schedule Appointment use case from Chapter 3, against the domain model from Chapter 4 and the architecture rules from Chapter 6. The request should be one sentence. Instead she opens a blank file, because she remembers what happened the last time she sent the short version: the assistant picked a testing library nobody on the team uses, logged nothing useful when a booking failed, and wrapped every error in a generic exception the rest of the codebase doesn't recognize.

So she starts typing everything the short version left out. Use JUnit 5 and Testcontainers for anything that touches the database. Log every appointment state change with its ID and the actor who caused it. Validate incoming requests with Jakarta Validation annotations, not hand-written checks. Wrap repository failures in a domain-specific exception, and return RFC 7807 problem-details responses for anything the API rejects. Use constructor injection, never field injection. Ten minutes in, the request is four paragraphs long, and she still isn't sure she's remembered everything the last three code reviews corrected.

CHAPTER PROMISE

By the end of this chapter, you can tell an implementation habit apart from a structural constraint, write that habit down once as a skill, and hand an AI assistant a request short enough to fit in a sentence, because the skill already carries the rest.

Reading time: 13 minutes

Where that paragraph came from, and why it keeps growing

Nothing about this is unique to one developer at Riverside Clinics. A colleague implementing the next feature writes nearly the same paragraph from memory a week later, and gets some of it wrong: she remembers Testcontainers but forgets the problem-details format, so the new endpoint returns a different error shape than the old one. A third developer never heard about constructor injection at all, because the rule lived in a comment on a pull request he wasn't copied on, so his service uses field injection and passes review anyway, because the reviewer that day didn't catch it either.

Chapter 6 was about rules nobody had written down at all: three developers naming the same kind of object three different ways because the naming convention existed only as habit. This is a different failure. These rules usually are written down, somewhere. A wiki page. A README nobody has opened since the project started. The knowledge exists. What's missing is anywhere the AI reads it at the moment the request is made, so every request re-states it, unevenly, from whatever each developer happens to remember that day.

A request that tries to specify testing, logging, error handling, and naming all at once, in the same breath as the actual feature, is a symptom: the knowledge it's straining to carry belongs to the project, not to whichever request happens to be open in whichever chat window that afternoon. Better wording won't fix that. Even a clearer version of the same paragraph still has to be rewritten, imperfectly, every single time.

The cost shows up gradually, then all at once: a support ticket traced back to an error format that only half the endpoints follow, a new hire who copies the wrong convention because it was the example nearest to hand, a quarter spent quietly reconciling five services that all did logging differently, none of them wrong exactly, just five different people's memory of a rule that was never written where any of them, or the AI helping them, could actually find it.

What a skill is, and what it isn't

A **skill**, in Simon Martinelli's usage, is reusable implementation knowledge: how an organization writes tests, handles errors, logs state, validates input, and names things, captured once and referenced by every feature that needs it instead of retyped into every request.

Chapter 6 drew a line between structural constraints, the ones that apply to every feature and rarely change (dependency direction, layering, module boundaries), and everything beneath that line. A skill lives beneath it. Architecture says a repository belongs in the infrastructure layer. A skill says what that repository looks like once it's there: which persistence library, which test harness, which convention for wrapping a failure before it reaches the service above it. Architecture is a constraint two teams cannot disagree on and still call themselves one codebase. A skill is a habit, and a codebase can hold the same architecture under several different, equally valid sets of habits, a claim this chapter's worked example will make concrete.

Chapter 6 also named a broader shift this book keeps returning to: prompt engineering asks how to phrase a request; context engineering asks what the assistant should already know before the request arrives. A skill is one answer to that second question, scoped to implementation conventions the way architecture answers it for structural ones. The pattern this book keeps arriving at, an interpretation earned from watching the difference play out rather than a measured finding, is that a plainly worded request backed by a complete skill produces more consistent code than a carefully worded request backed by none.

> **Architecture constrains where code goes. A skill decides what it looks like once it's there.**

Where the Model Context Protocol fits, and why a skill keeps changing

Skills, as this book describes them, are documents: written, reviewed, versioned, attached to a request instead of retyped into it. A related piece of infrastructure solves a neighboring problem by a different route. The Model Context Protocol, an open standard usually called MCP, lets an AI assistant query a live source, a coding standard, an API reference, a schema, at the moment it needs that information, rather than carrying it into every conversation by hand. The two ideas share an aim, less knowledge repeated per request, but not a mechanism: a skill is something a team writes and owns; MCP is a channel for retrieving something already written elsewhere. Which one an organization reaches for is a tooling decision. The discipline this chapter argues for, keeping implementation knowledge out of the request and in the project, holds regardless of which mechanism delivers it.

A skill is also not a document written once and shelved. When Riverside's platform team adopts a new logging library, they update the Spring Boot skill, not the memory of every developer who might ask an AI assistant to write a service next month. The next feature built against that skill inherits the change automatically, and the one after that, and the value of having written it down at all compounds with every feature built against it since.

Writing a skill instead of retyping it

1. Collect what keeps getting corrected. Look at the last several rounds of review feedback on AI-generated code for one stack, and list every correction that wasn't specific to a single feature: a testing library, a logging format, an error-wrapping convention someone keeps having to point out.
2. Separate habit from constraint. Anything the architecture already governs, layering, dependency direction, naming boundaries, stays in the architecture document. A skill holds only what's left: how to build within those boundaries, not where the boundaries sit.
3. Scope the skill to one stack. A Spring Boot skill and a Quarkus skill answer the same questions differently. Folding both into a single document produces a document that contradicts itself depending on which feature happens to read it.
4. State each convention as an instruction, not a principle. Not "handle errors well," but "wrap repository failures in a domain exception and return an RFC 7807 problem-details response." Specific enough that two developers implementing it independently converge on the same shape.
5. Attach the skill to the request instead of the paragraph. The request becomes a sentence naming the use case and the skill; the skill supplies everything the paragraph used to carry, and carries it the same way every time.

The Riverside Clinics booking feature, implemented against a written skill

Here is what this chapter's method produces for Riverside Clinics: a skill document the Spring Boot team keeps in the same place they keep the domain model and the architecture rules, referenced by name rather than retyped into a chat window.

Spring Boot implementation skill: Riverside Clinics booking service

Persistence. Spring Data repositories, one per aggregate root, the entity that owns a cluster of related objects (AppointmentRepository, not a generic data-access object). A repository never returns an entity straight to a controller; it's mapped to a data-transfer object first.

Dependency injection. Constructor injection only; a field-injected dependency is a review failure.

Validation. Jakarta Validation annotations on request objects (@NotNull, @Future on a requested slot time). Rules that depend on other data, like the 90-day booking window from Chapter 3, are enforced in the service layer, not the annotation layer.

Logging. Structured JSON logging through SLF4J, the Simple Logging Facade for Java. Every appointment state transition logs the appointment ID, the previous status, the new status, and the actor, patient or staff, that caused it. A patient's name or date of birth never appears in a log line.

Testing. JUnit 5. Any test that touches the database runs against Testcontainers, not an in-memory substitute. One integration test per business rule in the use case, named for the rule it checks: schedulesWithinBookingWindow_succeeds, not test1.

Error handling. A single controller-advice class translates domain exceptions into RFC 7807 problem-details responses. Repository failures are wrapped in a domain exception before they reach the service layer; nothing above persistence ever catches a raw database exception directly.

The same use case, a different skill

Send the same use case, the same domain model, the same architecture rules, to a team building the equivalent service on Quarkus instead, and only the skill changes.

> **Quarkus implementation skill: the same use case, a different stack**
>
> **Persistence.** Panache active-record entities, not repositories. An Appointment class extends PanacheEntity and persists itself directly.
>
> **Validation.** Hibernate Validator annotations, the same intent as the Jakarta Validation rules in the Spring Boot skill, applied on the entity rather than a separate request object.
>
> **Logging.** Unchanged: the same structured JSON format and the same required fields as the Spring Boot skill.
>
> **Testing.** JUnit 5 with the Quarkus test extension, plus a native-image build verification step the Spring Boot skill has no equivalent for.
>
> **Error handling.** One exception-mapper class per exception type, producing the same RFC 7807 shape the Spring Boot skill produces, arrived at through a different mechanism entirely.

Both implementations satisfy the same use case, the same domain model, the same rule that a patient can't double-book a slot. Both carry out identical business logic. Only the shape of the code differs, and that shape is exactly what a skill, not a use case, decides.

Where skills go wrong

▲ **WARNING**

A common way a team reaches for consistency is one file, STANDARDS.md or something like it, that starts reasonable and never stops growing. A rule gets added for the payments feature. Then one for the reporting module. Then a security exception discovered during an audit. Nothing gets removed, because nobody's certain what's still necessary. A year or two later that file is thousands of lines long, loaded in full on every request no matter what the request is about, and it contradicts itself in at least two places nobody has caught yet.

One instruction file for everything	A reporting feature loads payment-gateway conventions it will never use, and the one rule actually relevant to it sits buried on line 640.
A skill written per feature instead of per stack	Fifteen near-identical documents, each restating the same testing convention a single Spring Boot skill should have owned once, drifting slightly further apart with each new one.

Field guide: a worked skill definition

- ☐ Persistence: which repository or entity pattern, and what is a repository never allowed to return directly to a caller?
- ☐ Dependency injection: which injection style is mandatory, and what happens at review when someone uses another one?
- ☐ Validation: which library or annotation set, and which rules belong in the service layer instead of the framework?
- ☐ Logging: what format, what must appear on every state change, and what must never appear, such as a patient's personal data?
- ☐ Testing: which framework, which category of test is mandatory for which kind of code, and what naming convention marks which rule a test checks?
- ☐ Error handling: which exception-wrapping convention, and what shape does a client-facing error response take?

Fill in one answer per line for one stack your team actually uses. A document that leaves any line blank is a skill your team is still relying on someone's memory for.

What to remember

- A request that tries to specify testing, logging, error handling, and naming all at once is evidence that the knowledge it's carrying has nowhere else to live.
- A skill is reusable implementation knowledge, how an organization tests, logs, validates, and handles errors, kept separate from architecture's structural constraints.
- The same use case implemented against two different skills produces two different, equally correct implementations; the business logic stays fixed, and only the shape of the code carrying it out changes.
- One large instruction file feels like consistency and behaves like clutter. Several small, stack-specific skills scale in a way the giant file never does.
- A skill is a living document. Update it once, and every feature built against it afterward inherits the change without anyone retyping anything.

QUESTIONS FOR YOUR TEAM

- How many of your team's current AI prompts repeat the same testing or logging conventions, worded slightly differently each time?
- If your organization changed its error-handling convention tomorrow, how many existing prompts would need to be rewritten by hand before the new convention actually took effect?

ONE ACTION TO TAKE NEXT

Pull the last three feature requests your team sent an AI assistant for the same stack. List every implementation convention that shows up in more than one of them. That list is the first draft of your first skill.

TRANSITION. Vision, requirements, use cases, a domain model, and now architecture and skills: everything an assistant needs before it writes a line is written down. The next chapter follows all six layers through the rest of the workflow, from a finished specification to software running in production.

Skills: reusable implementation knowledge ends here.

8

PART 2: FRAMEWORK

From specification to software

What does the complete workflow from specification to deployed software look like in practice?

Riverside Clinics sets one go-live date for its booking feature: the Monday its marketing team emails twelve thousand patients that they can now book online. Two engineering teams are building toward that date. One is the team this book has followed since Chapter 1, the team that spent six chapters turning a one-line request into a vision, requirements, a use case, a domain model, an architecture, and an implementation skill. The other belongs to a physiotherapy group Riverside acquired six weeks earlier, and it received the same one-line request Riverside's own team got at the very start: let patients book appointments online, nothing else specified. Both teams have the same AI assistants. Both teams are competent engineers. What they don't have, going into the same six weeks, is the same amount of work already done.

CHAPTER PROMISE

By the end of this chapter, you can run a feature from a finished specification to a deployed release using the same ten-stage workflow, and tell which of its stages a shortcut is safe to take and which one never is.

Reading time: 13 minutes

What the same deadline actually measured

The acquired team opens an assistant on day one and has a working prototype by lunch. It looks like a head start. Doctors can add themselves, patients can pick a time, appointments save to a database. By week three, the prototype has been rebuilt four times. Doctors can double-book patients. Nobody on the team can say with confidence whether a patient with two email addresses on file is one person or two, because nobody decided, and the assistant decided for them, differently, in two different files. Every fix it produces is correct on its own terms and breaks something the last fix depended on.

Riverside's own team opens an assistant on day one too, and spends most of that day reading rather than typing. It reviews the specification Chapter 5 assembled, updates one domain-model detail a stakeholder conversation flagged as stale, and asks the assistant to implement a single use case against an existing domain model, architecture, and skill. Four days later, the feature is in review. The rest of the six weeks goes to two more use cases and to hardening what already works.

Both teams are running the same model, from the same vendor, on the same day. The four days and the three-week spiral measure something else entirely: how much of the business decision was settled before either team typed a prompt. Chapters 2 through 7 spent their effort building that settlement, one layer at a time: a vision, requirements, a use case, a domain model, an architecture, a skill. This chapter is about what happens once all six exist, and a team still has to turn them into a feature that runs in production by Monday.

The workflow that replaces improvising

Having a specification turns delivery into an ordered sequence rather than a single prompt: ten stages, each answering a question the stage before it couldn't, each removing a decision an AI assistant would otherwise have to invent on its own.

1. Vision: why the feature exists, and for whom
2. Requirements: what the system must do, observably
3. Use case: the complete interaction, step by step
4. Domain model: the business objects the feature runs on
5. Architecture: the structural rules the feature must fit
6. Skill: the implementation conventions the team already has
7. Implementation: the assistant translates stages 1 through 6 into code
8. Testing: the code is checked against what the use case actually said
9. Review: a human checks business correctness and consistency
10. Deployment: the feature ships through the pipeline that already exists

Chapters 2 through 7 built the first six stages for Riverside's booking feature. That work does not repeat itself for every new feature; a vision, a domain model, and an architecture are written once and reused across many use cases. What does repeat, every time a team turns a use case into working software, is the last four stages, implementation, testing, review, and deployment, plus one step this chapter adds before any of them, confirming the first six stages are still current.

> **The stages that take the longest happen once. The stages that happen every time are the ones a team can now run in days instead of weeks.**

Running the workflow once a specification exists

This is the cycle a team runs for every use case, once the vision, requirements, domain model, architecture, and skill it depends on are already written. It is short enough to run in days, and none of its five steps is optional.

1. Review the specification for staleness before implementing anything. Confirm the vision, requirements, use case, domain model, architecture, and skill still match what stakeholders agreed most recently, and update whichever one doesn't.
2. Ask the assistant to implement one use case against the existing domain model, architecture, and skill, and nothing beyond them. A specification-shaped request stays short because the decisions it depends on already live somewhere the assistant can read.
3. Generate tests from the use case's own scenarios: one test per step in the main flow, one test per alternative flow, so the tests and the specification describe the same behavior rather than two different ones.
4. Route the implementation to a human reviewer whose time is set by business risk, not by how many lines the assistant returned.
5. Deploy through the pipeline that already existed. Nothing about this workflow changes what continuous integration and delivery do once code reaches them.

Riverside Clinics: from specification review to working code

Here is Riverside's team running the first two steps of that cycle against the booking specification Chapters 2 through 7 built.

Step 1: specification review

```
The 90-day booking horizon from Chapter 3 hasn't changed. The domain model
has: clinic operations now lets a doctor block part of a day, not only a whole
one, and the model Chapter 4 wrote has nowhere to record a block that short.
The team adds one value object for it before writing anything else. Everything
else in the specification, the vision, the use case, the architecture, the
Spring Boot skill, is confirmed current and left untouched.
```

Step 2: implementation

```
"Implement the Schedule Appointment use case (Chapter 3) against the Riverside
Clinics domain model (Chapter 4, plus the new value object for partial-day
blocks) and the Spring Boot skill (Chapter 7)."
```

```
The request is one sentence, because every decision behind it already lives
somewhere the assistant can read. Minutes later, the feature exists: a booking
endpoint, a persistence layer, validation against the 90-day window and the
new partial-day rule, a confirmation notification.
```

Nothing in these two steps required the team to explain what a doctor is, what counts as an eligible time slot, or how Riverside logs an error. Chapters 3, 4, and 7 already answered those questions, once, for every feature that comes after this one.

Riverside Clinics: from generated tests to a deployed feature

Step 3: test generation

Each step of the use case's main scenario becomes a test. "System re-checks that the slot is open, then reserves it" becomes a test that a conflicting appointment is rejected. One alternative flow, a selected slot no longer available, becomes a test that the system offers another selection rather than failing silently. The tests describe the same behavior the use case describes, because they were generated from it rather than from the code.

Step 4: human review

Simon Martinelli, whose specification-driven methodology this book draws on throughout, puts the standard plainly: review effort should scale with business risk, not with how much code the assistant produced. Booking touches a patient's schedule but not billing or a clinical decision, so the reviewer spends little time on the routine persistence wiring and most of it on the one case that matters here: two patients reaching for the same slot at once, and whether the confirmation that goes out afterward is the one that should have.

Step 5: deployment

The feature ships through the continuous integration and delivery pipeline Riverside already runs, unchanged by anything in this chapter. Deployment is where the four preceding steps prove they were done correctly.

Where the workflow breaks down

▲ **WARNING**

Some accounts of this workflow go further and claim that once implementation is fast, regenerating a whole feature from an updated specification becomes as routine as re-running a build, cheap enough that code stops being something a team protects and becomes something it simply reproduces. That is a forward-looking possibility, not a settled fact. Nothing in the Riverside case, and nothing in the case studies this book draws on, shows implementation time and regeneration time actually converging in practice. This chapter treats it as a question worth watching, rather than a plan to build a team's process around today.

Skipping specification review because the specification already exists	A team implements confidently against a domain model that's a version behind the business, and the resulting bug looks like a coding mistake when it was actually a stale rule nobody checked.
Reviewing every feature at the same intensity, regardless of what's at stake	A reporting screen gets the same scrutiny as a billing change, and the reviewer has no time left when a risky feature needs it most.

Field guide: the specification-to-software workflow checklist

- ☐ Vision confirmed current, or updated if it drifted
- ☐ Requirements still observable, testable, and matched to what stakeholders agreed most recently
- ☐ Use case covers alternative flows, not only the main scenario
- ☐ Domain model reflects today's business rules, not the rules as they stood when it was written
- ☐ Architecture and skill unchanged, or the change is deliberate and recorded
- ☐ Implementation requested against the use case, domain model, architecture, and skill, nothing more
- ☐ Tests generated from the use case's own scenarios, one test per flow
- ☐ Review effort set by business risk, not by lines of generated code
- ☐ Deployment run through the existing pipeline, unchanged
- ☐ Specification updated to match, if anything changed during implementation

Run this before calling a feature done. A feature that passes every item but the last one will drift from its own specification by the time the next change reaches it.

What to remember

- The gap between a team that ships in days and a team that spirals for weeks is preparation, not which AI model either one used.
- A specification reaches software through ten stages: six of preparation, and four that repeat for every use case.
- Testing exists to confirm the implementation matches the specification, not merely that the code runs without error.
- Review effort should scale with business risk, not with how much code an assistant returned.
- A feature is done when its specification is current enough for someone else to read it and understand what shipped, and why.

QUESTIONS FOR YOUR TEAM

- The last time your team's review took longer than the implementation, was the feature actually high-risk, or did every feature get the same scrutiny by default?
- If your specification and your deployed feature disagreed today, which one would your team trust?

ONE ACTION TO TAKE NEXT

Take the next feature on your team's board that already has a specification behind it. Run it through the five-step cycle in this chapter, in order, and note which step your team was already skipping.

TRANSITION. Every example so far has had someone who could still say what the business meant: a stakeholder to interview, a habit to write down, a convention already sitting in a README nobody had opened. Most engineers spend their careers on systems where nobody left can say, and the only description of what the software does is the software itself. The next chapter is about recovering one.

From specification to software ends here.

9

PART 3: PRACTICE AND IMPLICATIONS

Brownfield development

How do you recover a usable specification from a system that never had one written down?

A developer at a retail bank picks up a change request that sounds routine: give customers a same-day grace period before an overdraft fee posts. The bank's core ledger has run since the late 1980s, written in COBOL by architects nobody currently on staff has ever met. She opens the module that calculates fees and starts tracing where the number actually gets decided.

AcctMaintenance calls BalanceEngine. BalanceEngine calls FeeSchedule. FeeSchedule calls a routine nobody has touched since sometime in the early 2000s, and three screens down she finds the line she's looking for: `IF ACCT-STATUS-CD = '07' THEN WAIVE-FEE`. Seven. Not documented in the code, not in the requirements binder from the system's early years, not in anyone's memory she's asked so far.

This is **brownfield development**: building on a system that is already live, already trusted with somebody's money, and already harder to read than the people who wrote it ever intended. Most software careers happen here, on systems built years or decades before, not on the blank slate every conference talk assumes. The code is the one thing that reliably survived. Whatever the business meant by it did not.

CHAPTER PROMISE

By the end of this chapter, you can treat an undocumented system as a specification waiting to be read, not only a codebase waiting to be replaced, and use AI to surface what its behavior implies about the business without asking it to decide anything on its own.

Reading time: 13 minutes

Where the missing knowledge actually goes

Code tells you how a system behaves. It rarely tells you why, and the gap between those two things is where the developer above will spend most of her week. She can find status code 7 in an afternoon; grep is good at that. Finding out what it meant, and whether waiving the fee for that status was ever a considered decision or just the behavior a fix from twenty years ago happened to produce, takes days of asking people who may no longer work there.

That search is the real cost of a missing specification, and it compounds. A requirement that would take an afternoon to confirm against a written use case instead takes a week of tracing calls, reading commit history nobody wrote useful messages for, and interviewing whoever has been there longest. Multiply that by every change request a legacy system receives over a decade, and the tax adds up. By the bank's own account, reported in Appendix A, adding any feature took six to eight weeks, and the tracing came before any of the building.

It gets worse than slow. The bank had tried, once before this chapter's scene, to replace the COBOL ledger outright: translate it line for line into a modern language, keep the behavior, retire the mainframe. Partway through, the team building the replacement noticed it didn't match the original in the edge cases, rounding here, an ordering difference there, and couldn't tell whether the mismatches were bugs to fix or business rules to preserve. Nobody left at the bank could say for certain. The project stalled on a question no amount of translation could answer: translation preserves syntax, not intent, and intent was the part nobody had written down.

Business knowledge that lives only in code slowly dissolves into implementation detail, until the only thing left of a rule is its side effect.

Reading a system as a specification, not a debugging problem

Specification recovery is the practice this chapter is about: reconstructing a system's vision, requirements, use cases, and domain model from what its code and its people currently do, instead of writing those layers from a blank page the way the rest of this book has, so far, assumed you could. It runs the specification-driven method in reverse. Rather than starting from a decision and generating code, you start from code and work backward to the decision that, at some point, someone made.

Martin Fowler already named a version of this discipline, applied to architecture rather than knowledge. He called it the Strangler Fig pattern: intercept calls to one piece of old functionality at a time, route them to a new implementation, and retire the old piece once nothing depends on it anymore. It's an established, widely used way to replace software gradually instead of all at once, and it works because it never asks a team to understand the whole system before touching any of it.

Specification recovery borrows that discipline and points it one layer earlier. Instead of replacing running code piece by piece, you recover the knowledge behind one piece of it, validate that knowledge with someone who can vouch for it, and only then decide what should replace the code. Call this chapter's version an extension of Fowler's logic, not a restatement of it: his pattern strangles code without ever requiring anyone to understand what it was for, while this one only works if someone does.

> **Code can be replaced by a team that never understood it. A specification can only be recovered by people who do, which makes recovery the slower half of the work and the half that decides what survives the migration.**

Recover, validate, improve, then generate

A straight line-for-line migration asks one question: does the new code do what the old code did? Specification recovery asks a different one first, and defers the migration until it has an answer.

1. Recover. Point an AI assistant at one module, one workflow, or one integration at a time, and ask it to describe, not to fix. Entities, business rules, validation logic, the shape of a use case, all of it read out of the code as it stands today.
2. Draft the specification layers from that description, flagging anything that looks like suspicious complexity, duplicated logic, a magic number, a rule with no visible reason, rather than resolving it. This is where AI's role stays archaeological: it surfaces what's odd, it does not decide what the odd thing was for.
3. Validate every recovered rule with a person who has standing to confirm or dispute it, an operations lead, a compliance officer, whoever has lived with the system's actual behavior. Some rules will be confirmed instantly. Others will start an argument, and that argument is the point.
4. Only once a rule is confirmed as intentional, or corrected as a mistake nobody meant to keep, generate the new implementation against it. Improve the architecture at this stage, not the business meaning; that decision was already made in step three.

Recovering "process a withdrawal" from the bank's ledger

Back to the developer and her overdraft fee. She and an AI assistant work through FeeSchedule and the two modules that call it, and instead of asking for a fix, she asks for a description: what does this code actually do, rule by rule, before anyone decides whether it's right.

The first two rules come back uncontroversial. A withdrawal that would take the available balance below the account's minimum is declined unless an overdraft line is attached; everyone who's touched the ledger recognizes that one on sight. Status code 7 takes longer, but an operations lead who has supported the system for over a decade remembers it: a hardship flag, set by a case worker for a customer in an approved financial-hardship program, waiving the overdraft fee for as long as the flag is active. Confirmed, and now written down for the first time.

The third rule is where the recovery stops being tidy. The code always posts same-day deposits before same-day withdrawals, regardless of which one the customer submitted first. A compliance analyst reviewing the draft specification flags it immediately: transactions should process in the order they arrive, and this looks like a defect that happens to favor the customer. The operations lead disagrees just as fast. That ordering, she says, has kept thousands of accounts out of overdraft for as long as she's worked there; change it, and the bank will hear about it from customers before the week is out.

Neither of them is wrong about what the code does. They disagree about what it means, and that is not a question AI can settle by reading more code. It goes into the recovered specification as a flagged decision, not a resolved one.

> **Recovered use case: process a withdrawal (draft, unconfirmed)**
>
> Primary actor: retail account holder, via the ledger's nightly batch.
>
> BR-1: Decline if available balance minus the withdrawal falls below the account minimum, unless an overdraft line is attached. Confirmed.
>
> BR-2: Waive the overdraft fee when account status is 07 (hardship flag, case-worker set). Confirmed by operations; undocumented anywhere else.
>
> BR-3 [DISPUTED]: same-day deposits post before same-day withdrawals, regardless of submission order. Compliance reads this as a defect. Operations reads it as fifteen years of protecting customers from overdraft fees. Recovered as-is. Decision owner: head of retail operations.

What this work costs when a specification was never missing

Riverside Clinics has no equivalent of status code 7, and the difference is worth sitting with. Say the clinic group needs to move its booking system off its current stack ten years from now, to whatever platform has replaced it by then. Nobody will need to reverse-engineer why a slot only counts as available when the doctor works at that location, or where the 90-day booking horizon came from, because chapter 5 wrote both down as REQ-07, in the same document as the use case that applies them, and chapters 6 through 8 kept it current as the system grew.

The work in that future migration looks nothing like this chapter's. Someone opens the versioned specification, checks it against what the system currently does, updates what's changed since the last version, and hands the result to AI for the new stack. No archaeology, because nothing was buried in the first place. No dispute between operations and compliance over what a rule was always supposed to mean, because the rule was written down by the people who made the decision, not inferred years later from its side effects.

THE DIFFERENCE IN ONE LINE

The bank is paying, decades late, for a specification it never wrote. Riverside is spending a little now so that a future migration would have nothing to recover, only something to carry forward.

Where recovery goes wrong

▲ **WARNING**

Everything AI recovers from a legacy system is a hypothesis about intent, not a fact about it, until a person with standing to know confirms or corrects it. Treating a recovered rule as settled the moment it's written down skips the one step that makes recovery different from guessing with extra steps.

Trusting a recovered rule without validating it	A team builds a new implementation against an AI-inferred rule, ships it, and later learns the rule was an edge-case workaround someone patched in once, not a general policy anyone intended to keep.
Treating recovery as a one-time project	A team recovers a specification, generates a clean new system from it, and never updates the specification again. Five years and a dozen unrecorded changes later, it is exactly as unreliable as the code it replaced.

Field guide: a specification-recovery checklist

- ☐ Has every recovered business rule been traced to a person who can confirm or dispute it, not only to the line of code it lives in?
- ☐ Is anything AI flagged as suspicious logged for a human decision, rather than quietly changed or quietly kept?
- ☐ Does the recovered specification distinguish behavior confirmed intentional, behavior confirmed accidental, and behavior nobody has judged yet?
- ☐ Would a stakeholder who has never read the original code recognize the recovered use case as their own process?

Run this before generating any new implementation from a recovered specification. An unconfirmed rule that reaches production is a bug wearing the credibility of a written specification.

What to remember

- Most software careers are spent maintaining systems that already exist, not building ones that don't; brownfield work, not greenfield, is the default.
- Code records how a system behaves. It does not reliably record why, and business knowledge stored only in code eventually loses its meaning entirely.
- AI's most useful role in a legacy system is archaeological: surfacing suspicious complexity for a human to judge, not rewriting anything on its own authority.
- A recovered rule is a hypothesis until someone with standing confirms it. Two people can read the same behavior and disagree about what it was always supposed to mean, and that disagreement belongs to the business, not to the code.
- Recover, validate, improve, then generate, in that order. Skipping straight to generation just automates the guessing a manual rewrite would have done anyway.

QUESTIONS FOR YOUR TEAM

- Which system your team maintains has the most undocumented business rules buried in its code, and who is the one person left who could still confirm what any of them mean?
- If that person left tomorrow, how much of what they know would still be recoverable?

ONE ACTION TO TAKE NEXT

Pick one module of a legacy system your team maintains. Ask an AI assistant to describe, not fix, what it does, rule by rule, then take that description to the one person most likely to confirm or dispute it.

TRANSITION. Recovering a specification answers what a legacy system does. It doesn't answer what happens to the engineers who used to be the only ones who knew. The next chapter turns from the systems to the people who build and maintain them.

Brownfield development ends here.

10

PART 3: PRACTICE AND IMPLICATIONS

The future engineer

How does the day-to-day work of a software engineer change once implementation is largely automated?

Three engineers on the Riverside Clinics team open the same finished specification: the Schedule Appointment use case from Chapter 3, the domain model from Chapter 4, the Spring Boot skill from Chapter 7. Each points an AI assistant at it separately. Their team lead expects three nearly identical pull requests back within the hour. A use case this precise, she assumes, should produce one obvious answer.

It doesn't. All three implementations pass every test the use case specifies. None breaks a business rule. And no two of them handle a patient trying to book an already-taken slot the same way. One locks the slot in the database the instant a patient starts confirming it. Another lets several patients see the same open slot and resolves the conflict only when someone commits. The third holds the slot for 90 seconds while the patient finishes confirming, then releases it if they don't. The lead's first instinct is that something is wrong. Nothing is wrong. The specification did exactly what it was written to do, and there was still a decision left for a person to make.

CHAPTER PROMISE

By the end of this chapter, you can name what moved from scarce to automated when AI took over implementation, what stayed scarce despite it, and place your own current skills somewhere on the line between the two.

Reading time: 12 minutes

A question every abstraction has provoked

The question the Riverside engineers are really answering, without quite phrasing it this way, is one software has asked before. When compilers replaced hand-written assembly, people asked whether programmers would still be needed. When fourth-generation languages promised that analysts could generate applications directly from specifications, the same question came back under a new name. Low-code platforms revived it again later: would business users simply build the software themselves, no engineer required?

None of those transitions removed the engineer. Each one moved what engineers spent their day doing, away from whatever the new abstraction had just made cheap and toward whatever stayed scarce once it had. That's a pattern repeated across decades of software history, not a guarantee about what happens next. AI-assisted implementation goes further than any of the three before it, though. It doesn't just remove one layer of manual translation. Handed a finished specification, it can produce a working, tested implementation directly. What matters is what it still leaves for an engineer to do, and the scene at the start of this chapter already contains the answer.

What got cheap, and what didn't

Call the first half of the old job **syntax fluency**: knowing the keywords, the framework calls, the correct way to wire a repository to a service to a controller. For most of software's history, syntax fluency was the scarce half. An organization paid for people who could reliably produce it, and reliably producing it was most of what separated a senior engineer from a beginner.

AI assistants made syntax fluency cheap. Given the Schedule Appointment specification, an assistant produces syntactically correct, idiomatic, tested code for it in minutes, in whichever stack the project's skill tells it to use. What an assistant cannot supply is the other half of the job: **judgment**, this chapter's term for knowing which of several syntactically correct implementations serves the business, and being able to say why. The three engineers at Riverside all had syntax fluency. AI made that part cheap for all of them. What separated their pull requests was judgment, and judgment is exactly what stayed scarce.

Judgment, in turn, draws on something specific: knowing the business well enough to notice when a specification's silence matters. An engineer who has spent two years on Riverside's scheduling system knows that clinic operations cares more about a doctor's calendar never double-booking than about shaving a few hundred milliseconds off a booking request. A generically skilled engineer, however fast at producing syntax, has to ask before they know that, or guess. That advantage, domain knowledge over syntax fluency, is new only in how much it now outweighs the other kind.

> **Syntax got cheap. Judgment didn't.**

What stays human regardless

Judgment isn't the only thing this abstraction doesn't reach. Four capabilities in particular don't show up in a specification, however well it's written, because a specification records what they produce, not the work of producing it.

Negotiation is one: getting the clinic's operations team and its front desk staff to agree how much notice a cancellation needs, before that agreement becomes the 24-hour rule in Chapter 3's use case. Ethics is another: deciding what the system should do when the technically correct answer harms a patient in a way no test case anticipated. A third is leadership, getting three engineers with three defensible implementations to agree on which one ships, then getting the team to move forward once that decision is made. The fourth is specific domain expertise: knowing, because you've sat in the room, that clinic operations chooses predictability over speed, before anyone has written it down.

None of the three engineers' pull requests needed any of that to be syntactically correct. All three needed it to be chosen.

Reviewing what a specification left open

Once syntax is cheap, review work moves with it. Checking whether generated code matches the codebase's formatting conventions used to take time; that check is worth almost nothing today, since an assistant following the project's skill gets it right by default. What review is for now is the question none of that touches: does this implementation still mean what the specification said, and where it diverges from another correct version, was that an oversight or a deliberate design choice?

1. Confirm the implementation satisfies the specification's business rules and acceptance criteria. A passing test suite checks this; a reviewer's own reading of the use case against the code confirms it.
2. Find exactly where two spec-compliant implementations diverge. That divergence marks a place the specification was silent by design, an implementation detail, not a place it was violated.
3. For each divergence, name what it affects. Correctness is already settled at step one, so what's left is behavior under load, the patient's experience, or how hard the code is to change later.
4. Weigh those effects against something the specification never had to state, because it isn't a business rule: expected traffic, the team's operational maturity, where the business plans to take the feature next.
5. Choose, and write down why, in an architecture decision record or a comment on the pull request, so the next engineer, or the next AI run, inherits the reasoning and not just the code.

Three ways to hold a slot, on paper

Return to the three pull requests at the start of this chapter. Each implements the Schedule Appointment use case's core rule correctly: one patient, one eligible doctor, one open slot, inside the 90-day booking window agreed with clinic operations back in Chapter 3. Laid side by side, though, they make different bets about a question the use case never had to answer, because answering it isn't a business rule.

APPROACH	WHEN TWO PATIENTS REACH FOR ONE SLOT	WHERE IT WINS	WHAT IT COSTS
Lock at confirm	Database row lock held from the moment confirmation starts	Simple to reason about; the winning patient loses no work	The second patient stalls waiting on the lock, worse under high volume
Resolve at commit	Both patients proceed until commit; the loser sees "slot no longer available"	Scales well under contention; nobody waits	The losing patient's confirmation work is wasted and needs its own retry screen
90-second hold	First patient gets an exclusive hold while the confirmation is in progress	Matches how patients already expect travel booking to feel	Needs a background job to expire stale holds, more moving parts to operate

The specification doesn't say which of the three is correct. It shouldn't. Deciding between them needs information the specification correctly leaves out: how often two Riverside patients collide on the same slot, whether clinic operations wants the reassurance of a visible hold, whether the team already operates the kind of background job the third option needs. That decision belongs to the lead, and it's engineering judgment, not a gap in the specification or a flaw in what the AI understood.

Where this shift can mislead

▲ **WARNING**

Judgment being scarce doesn't make it automatic. A reviewer who waves through three implementations because a human chose one has skipped the judgment call, the same way a rubber-stamped syntax review used to skip catching a bug. The bar moved. It didn't disappear.

Treating this shift as license to relax review effort	A team stops asking which implementation the business needs and ships whichever pull request finished first, the exact shortcut this book has spent nine chapters arguing against.
Treating domain expertise as sufficient on its own	An engineer who knows the clinic's rules well starts deciding from memory instead of the specification, reintroducing the undocumented, in-someone's-head knowledge that made brownfield systems (Chapter 9) hard to recover in the first place.

This chapter is on solid ground describing what has already moved: syntax cheap, judgment scarce, review's attention shifting toward intent. It's on thinner ground guessing at exactly what that produces ten or twenty years out, a specific curriculum, a specific job title, so treat what follows as one plausible sketch, not a forecast with a date attached. A curriculum built heavily around requirements engineering, domain modeling, and what Chapter 6 called context engineering is one reasonable way engineering education could respond to this shift. A title like "knowledge engineer" is one name someone might give the resulting role. Neither is a prediction this book is making; both illustrate the pattern the chapter argues for, that the valuable half of the job is shifting from producing syntax to organizing and validating knowledge.

Field guide: where your skills sit on the syntax-to-judgment line

- [ ] If your AI assistant were unavailable for a week, could you still produce syntactically correct code for your stack, or would you only recognize whether the assistant's output was correct?
- [ ] When two AI-generated implementations of the same use case both pass every test, can you name the trade-off that should decide between them for your team, not a generic best practice?
- [ ] Do you know the business reason behind a specific field, status value, or edge case in your system without asking someone else first?
- [ ] In your last three code reviews, were most of your comments about formatting and style, or about whether the code matched the specification's intent?
- [ ] Could you write the specification for a feature you maintain regularly, from scratch, without an existing one to copy?

Answering "no" to most of these points at what's worth building next: domain knowledge and specification judgment, not more syntax practice.

What to remember

- Every prior abstraction, compilers, fourth-generation languages, low-code, raised the same fear about engineers disappearing. The engineers never disappeared, and each shift moved what they spent their time on instead.
- AI made syntax fluency cheap. It didn't make judgment, the ability to choose well between several correct implementations, any less scarce.
- Domain knowledge now outweighs syntax speed by a wider margin than it used to, because the syntax half of the job got automated and the judgment half didn't.
- A specification can be entirely correct and still leave more than one valid implementation standing. Choosing between them, and explaining why, is still engineering work.

QUESTIONS FOR YOUR TEAM

- When someone on your team chooses between two correct implementations, where does that reasoning get written down, and who reads it later?
- If two engineers on your team produced different, both-correct implementations of the same use case tomorrow, who would be equipped to choose between them, and why?

ONE ACTION TO TAKE NEXT

Pull the last pull request your team accepted without much discussion, one that came back from an AI assistant fast and clean, and ask what would have been different about it if two other engineers had implemented the same specification separately. If you can't answer, that's the skill this chapter is arguing you build next.

TRANSITION. This chapter has looked at what changes for the engineer. The next chapter asks the same question about the organization: once specifications are what a business protects and reuses, and code is only one disposable expression of them, what changes about how software competes?

The future engineer ends here.

11

PART 3: PRACTICE AND IMPLICATIONS

The specification economy

What changes in software economics once specifications, not code, are the asset that persists?

At the retail bank from Chapter 9, the ledger's second migration arrives sooner than anyone expected. Not away from COBOL this time. COBOL is long gone, replaced years earlier by the Java service the recovery project built. Java is now the system under review. A newer platform the bank's infrastructure team has been trialing runs the nightly reconciliation job at a fraction of the cost, and finance wants the migration scoped by next quarter.

The engineer assigned to scope it does something her predecessor could not. She opens a specification, not a codebase. The reconciliation rule that project recovered from thirty-year-old batch code sits on her screen in the form it was left in: actor, business rule, postcondition, updated twice since through ordinary change requests, never rewritten from scratch. She does not page through Java source hunting for where a mismatch gets flagged. The rule is already written down, dated, and versioned. What took a recovery project the first time takes an afternoon the second time.

CHAPTER PROMISE

By the end of this chapter, you can name what your organization's software economics actually reward once a specification outlasts the implementations built from it, and identify where your own team's business knowledge currently lives only in code nobody plans to reopen.

Reading time: 12 minutes

What the industry has spent decades protecting

For most of computing history, source code has been the asset organizations protect. Non-disclosure agreements guard it. Source-code escrow agreements insure against a vendor's collapse. Acquisitions price a company partly on the size and condition of its codebase. All of that spending rests on an assumption nobody examines closely: that the code is what makes the business hard to copy.

Run the thought experiment anyway. If a competitor obtained your entire technology stack overnight, every repository, every deployment script, would they be running your business tomorrow? Almost certainly not. They would have syntax. They would not have the decisions the syntax encodes: which exceptions to a rule the business actually tolerates, versus which ones only look tolerated because nobody ever enforced them; why a price calculation rounds the way it does; which regulatory workaround from a decade-old ruling is still load-bearing. None of that survives a copy-paste. Chapter 9 already showed why: the meaning of `ACCT-STATUS-CD = '07'` survived only in one operations lead's memory, and the deposit-ordering rule beside it had no agreed meaning at all.

The consequence is a mismatch. The organization spends its protective effort, legal, security, contractual, on an asset that would not reproduce the business by itself. Meanwhile the asset that would, the accumulated decisions behind the code, sits wherever Chapter 9 found it: in a departing employee's memory, in a batch job nobody has touched since the early 2000s, in no document at all.

The asset computing keeps moving

Software has organized itself around a different scarce resource in each era. In the mainframe decades, the asset was hardware: owning a machine capable of running a payroll job was the competitive edge. Once machines became common, the asset moved to software itself, applications expensive enough to write that owning the source code mattered. Then it moved again, to platforms: an operating system, a cloud provider, a marketplace, the layer everyone else's software had to run on top of. Each shift didn't erase the one before it. Hardware still matters. Software still matters. Each shift did move where the money and the advantage concentrated.

This book's term for where that concentration is heading next is the **specification economy**: an arrangement in which the specification, not the code generated from it, is the asset an organization protects, versions, and reuses, because the specification is what survives every rewrite. That name is a proposal this book is making, not a category the industry has already settled on. What supports it is the pattern this book kept running into, most visibly in the bank's ledger, where the rule recovered once has already outlasted the code it was recovered from.

> **Code executes a decision. It does not explain the decision it is executing, and whoever holds only the code has to reconstruct the one thing that was never written down in it.**

Finding out which asset your organization actually protects

The question this chapter is asking, restated at the level of a single team, is checkable in an afternoon.

1. List the systems where replacing the technology keeps turning into a multi-year project instead of a straightforward rewrite. Not every brownfield system qualifies, only the ones where the cost is understanding rather than translation.
2. For each one, ask what would actually be lost if the code disappeared: the syntax, or the understanding behind it. If a departing team could hand the next team a specification instead of a repository, and the next team could implement it correctly, the organization's real asset is that specification, whether anyone has called it one yet.
3. Compare that list against what the organization currently budgets to protect: security review, access control, escrow, backup policy. Wherever the two lists disagree is where the protection spend is pointed at the wrong asset.

One rule, three implementations

The bank's ledger is this chapter's case for a reason. It is the one running example in this book old enough to have already outlived a technology generation, specific enough that the outliving is checkable rather than asserted. The rule below came out of Chapter 9's recovery project, recovered from the ledger's nightly batch job alongside the withdrawal rules that chapter walks through. Nothing about its wording depends on the language that ends up executing it.

> **Reconcile Daily Ledger: recovered business rule**
>
> `Actor: nightly reconciliation job, unattended.`
>
> `Business rule: an account's ledger balance and its transaction history must sum to the same value at close of business. A mismatch of any amount holds the account for manual review before the next posting cycle.`
>
> `Postcondition: every open account either reconciles automatically or is flagged with the specific transaction range under review, never left unreconciled overnight.`

THREE IMPLEMENTATIONS, ONE RULE

- **1987, COBOL:** the rule as originally written, recoverable only by reading the batch job line by line. No other record of it survived.
- **2019, Java:** the rule as the recovery project captured it, the same mismatch-and-hold behavior, now stated as a specification instead of buried in a nightly batch script.
- **Next migration, platform undecided:** the technology under review changes; the specification does not. The team scoping the move starts from the rule above, not from Java source, and the rule stays testable regardless of what ends up running it.

Riverside Clinics never faced this test; its booking specification is only a few chapters old by comparison. The bank's ledger has, and so far the specification has held where the COBOL it was recovered from did not.

What stays speculative, and what a specification cannot yet do

▲ **WARNING**

Two ideas this chapter gestures toward are not yet working practice anywhere in this book's case material. The first is regenerable software: swap the implementation skill, regenerate the system on a new stack, specification unchanged. The second is specifications treated as protected intellectual property the way source code is now, with the legal and tooling support that implies. Both are plausible directions, consistent with everything else in this chapter. Neither is a documented practice yet, and this book will not claim otherwise.

This book uses the phrase **knowledge debt** for the pattern behind most of Chapter 9's recovery work: undocumented business rules, inconsistent terminology across systems, requirements nobody reconciled, assumptions a team stopped questioning. Knowledge debt is not an established industry term the way technical debt is. It's offered here as a name for something those cases kept describing without naming, not a category the field has already agreed on.

Treating "we wrote a specification" as the finish line	A badly written specification is worth as little as undocumented code. It just fails later, when someone tries to implement from it and can't.
Assuming legal protection for specifications works like it does for code	Copyright and trade-secret practice were built around source code. Nobody in this book's case material has a working answer yet for what protects a specification the same way.

Field guide: locating knowledge that lives only in code

- ☐ Could a new hire explain why this rule exists, or only what the code currently does?
- ☐ If the person who last touched this system left tomorrow, could someone else recover its reasoning within a week, or would it take a project like Chapter 9's?
- ☐ Do two systems in your organization implement the same business rule differently, because nobody wrote down which version is correct?
- ☐ Is there a rule your team enforces in code that no business stakeholder could check against a written description of it?

Three or more answers pointing to code as the only record mark a system where your organization's knowledge debt, proposed term or not, is concentrated.

What to remember

- Every prior shift in computing, hardware, then software, then platforms, moved what an organization treats as the asset worth protecting. Nothing here proves specifications are the next move; the bank's ledger is one case where the pattern has held so far.
- A stolen codebase does not reproduce a business, because what makes a business hard to copy survives as decisions, not as syntax a thief could read off a stolen repository.
- The bank's ledger specification recovered in Chapter 9 has already outlived one migration. This chapter's field guide is how to find the systems in your own organization where that hasn't happened yet.
- Regenerable software and specifications protected the way code currently is remain speculative. Treat them as a direction worth watching, not a workflow to adopt this quarter.
- This book's bet is that competitive advantage moves toward whoever understands their own business most precisely and has written it down, not toward whoever ships code fastest.

QUESTIONS FOR YOUR TEAM

- Which of your systems, handed to a brand-new team tomorrow with nothing but the source code, would take that team the longest to understand rather than to rebuild?
- Where does your organization currently spend the most effort protecting code that, by itself, would not let a competitor reproduce your business?

ONE ACTION TO TAKE NEXT

Pick one system from this chapter's field guide. Write, on a single page, the one business rule inside it that nobody could explain to a new hire without opening the code. That page is the first entry in a specification, whether or not it looks like one yet.

TRANSITION. That is the argument this book has made across eleven chapters, from a one-line request at Riverside Clinics through a COBOL ledger at the bank: write the specification, and the code that implements it becomes one of several replaceable answers rather than the thing the business depends on. The Afterword closes it out.

The specification economy ends here.

BACK MATTER

Afterword

The book has made its case. What's left is to say plainly what it doesn't cover, and what would have to be true for the case to be wrong.

By now you've either tried writing a vision statement for a feature your team was about to hand an AI assistant, or you've filed this book's exercises under someone else's project. Either way, the argument is finished. Eleven chapters built one example, a clinic group's booking feature, and stress-tested the method against a second, a bank's 1980s COBOL ledger, to show what a specification is for and what happens to a system that never had one written down. What's left in these last few pages is smaller: what the book actually claimed, what it left out on purpose, and the honest possibility that the whole thing rests on an assumption that doesn't hold.

What the argument comes down to

Set the examples aside and the claim underneath them is short. Deciding what the code should do has always cost more than typing it did, and AI made that visible only by removing the part that used to disguise it: writing the code doubled as thinking the problem through, for as long as a person had to write it by hand.

A specification, written down and kept current, holds that decision longer than the codebase built from it, because the technology underneath changes before the business logic does. COBOL gave way to Java in the bank's ledger. The rule that a ledger mismatch holds the account for manual review did not. Structure is what lets AI implement a decision instead of inventing one: vision, requirements, use cases, a domain model, architecture, skills. Structure is also what lets a human check the result against something more specific than a feeling.

This argument moves judgment in engineering away from syntax, which nobody has to type by hand anymore, and back toward the question that was always hard: what should this system do, and for whom.

Where this book stops short

A methodology book has to stop somewhere. This one stopped short of several subjects that deserve a book of their own rather than a section tacked onto this one.

Ethics and accountability: this book has assumed a specification with one accountable owner who stays responsible for it, and an implementation reviewed by a human before it ships. That's a workable model at the scale of one clinic group's booking feature. It says nothing about what changes once an organization is implementing hundreds of specifications a week and no single person has read all of them. This book doesn't answer that question.

Security and trust: generated code needs the same scrutiny as any other code, and in places more. An AI assistant filling in a validation rule it inferred, rather than one the domain model specified, is a plausible way for a vulnerability to enter a system that looks entirely conventional on review. This book mentions that risk without building the review discipline it requires.

Organizational change: adopting this discipline changes who signs off on a specification, what a standup reports on, and how a team that has always been measured on shipped code reacts to being asked to spend a week on a domain model instead. Those are people problems more than technical ones, and this book, being a technical methodology, was never equipped to solve them.

Tooling: git, diff, and code review all grew up around code. Nothing described here has an equivalent that's mature and widely adopted. A skill from Chapter 7 ends up as a document in a repository, the closest available container rather than the right one.

Regulatory compliance: the bank's ledger, this book's own running example, sits inside documentation and audit-trail requirements that a regulator enforces directly. Whether a specification recovered the way Chapter 9 describes would satisfy a specific regulator is a question for someone who knows that regulator.

What if the premise is wrong

Everything above assumes a bet pays off: that a specification is cheaper to keep current than code is, and that AI can be trusted to implement from one reliably enough that the review this book describes stays lighter than a full rewrite. That bet could be wrong.

Specifications could drift the same way code comments do, accurate on the day they're written and stale within two sprints, because nothing enforces keeping them current the way a compiler enforces syntax. A use case that says a patient books one slot with one eligible doctor is only as good as the discipline that updates it the day that rule changes, and nothing in this book manufactures that discipline out of nothing.

An organization could adopt every layer this book describes, vision through skills, and still end up maintaining two things instead of one: the specification, and the code that drifted away from it. That would be a worse position than maintaining code alone. This book cannot rule out that failure. It shows up exactly where the discipline is weakest, usually in the specification nobody has looked at since the meeting where it was written.

Chapter 8's advice on testing and Chapter 9's advice on legacy recovery both apply here just as much as they applied there: start on one feature, keep the specification and the code in view together, and treat a specification you've stopped trusting as a signal to repair the discipline. A pilot that fails this way is a finding, and reporting it honestly to the rest of the organization is part of the discipline working as intended.

Why this, why now, and where that leaves you

None of the ideas in this book are new. Engineers have written some version of a specification for decades, and the Foreword traces that lineage. What changed is the arithmetic. Hours not spent specifying the work show up later, as review time and rework, on code nobody can be sure was built for the right business.

Whether that arithmetic holds up in your organization depends on how often your requirements actually change, how much of your domain currently lives only in someone's head, and whether your team keeps a specification current once the novelty of a new practice wears off, the way it does for any practice.

What this book can offer is a specific, checkable test. The Schedule Appointment use case in Chapter 3, the domain model in Chapter 4, and the Spring Boot skill in Chapter 7 are things you can write for your own feature this week. If the specification you produce lets an AI assistant implement that feature correctly on something close to the first pass, the argument in this book has done its job for you. If it doesn't, you'll know quickly, and you'll know exactly which layer to blame.

That's the test this book leaves you, and it's a more useful measure than whether the argument was persuasive.

Afterword ends here.

BACK MATTER

Appendix A: Case studies

Five organizations that adopted aspects of specification-driven development, as each of them reported it, with the details that would identify them changed.

Every methodology has to survive contact with the messiness of a real organization: the founder who left before writing anything down, the vendor contract signed under deadline pressure, the department that trusts its own spreadsheet more than the shared system. The five cases in this appendix are drawn from a bank's core ledger migration, a clinic network's scheduling overhaul, an e-commerce team's shift away from ad hoc AI use, an enterprise's replacement of a standards document teams ignored, and a government agency's benefits system last touched in 1995. Between them, they cover most of what the earlier chapters treat one thread at a time: recovering a specification from code that predates everyone currently maintaining it, writing one from a blank page, and using a shared specification to keep several teams, or several technologies, pointed at the same behavior.

▲ WARNING

Before you read these as a track record: each case here is presented as reported by the organization involved, with names, locations, and other identifying details changed. Nobody involved supplied independently audited figures, and none of these five cases was selected to represent an average outcome.

Read every number in this appendix as what one organization says happened on one engagement, not as a guarantee or a typical result. A processing time cut from 45 days to 5, a review-time reduction of 60 percent, a rewrite that landed at a quarter of its original estimate: each is one organization's report of one engagement, under its own conditions. What you get from the same approach depends on your codebase, your regulatory constraints, and how much specification work your organization is willing to do before it starts generating code.

CASE 1 · FINANCIAL SERVICES

A line-for-line COBOL rewrite stalls, and a bank recovers a specification instead

A retail bank, the one whose ledger Chapters 9 and 11 follow, had run its core account system on COBOL written in the late 1980s: deposits, withdrawals, transfers, and account management for 200,000 customers. The architects who designed it had retired. Documentation was outdated. The system worked, but adding a feature took six to eight weeks and risked destabilizing everything around it, while fintech competitors shipped comparable features in a fraction of that time.

The bank's first plan was a straight technology swap: rewrite the COBOL in Java, move to the cloud, an estimated 18 months and $2.5 million. Three months in, the new system's behavior started drifting from the old one. Small differences in rounding. In the sequence operations ran in. In how edge cases were handled. Nobody currently at the bank fully understood the original rules; they existed only as COBOL. Some of what looked like rules turned out to be bugs that customers had learned to depend on, which meant fixing them would break customer workflows. The rewrite stalled.

The bank brought in specialists to recover a specification from the code instead of replacing it outright. Over eight weeks, working through the codebase and interviewing the senior staff who still remembered pieces of the original logic, they documented a vision, roughly 200 specific behavior rules, 15 major use cases, and a domain model of 8 core entities. Only then did they generate a Java implementation, validate it against 10,000 test cases pulled from production, and run the old and new systems side by side for three months before cutover.

REPORTED RESULT

- Six months and $600,000, against an 18-month, $2.5 million estimate for the direct rewrite
- Zero production issues at cutover, after three months running in parallel with matching output
- New-feature turnaround down to two to three weeks, from six to eight

The bank's own account credits the result to finally knowing what the system was supposed to do, independent of COBOL or Java. Technology became something they could change without losing the rules underneath it.

CASE 2 · HEALTHCARE

A clinic network abandons a stalled vendor integration and writes its own specification

A healthcare network of 40 clinics and 500 doctors had a different scheduling system at almost every location, each built by a different vendor. Patients couldn't book across clinics. Doctors entered their availability by hand, sometimes into more than one system. Double-bookings were routine, and the network was losing patients to competitors with simpler booking.

The CTO's first move was to buy a single enterprise appointment system: one vendor, one platform, a quoted $3 million and 12 months. Six months into integration, the plan was straining. Every clinic had its own business rules buried in its old workflow. The vendor's platform was flexible enough to accommodate them but complicated enough that different parts of the network configured it differently, and the inconsistencies were multiplying rather than resolving.

Instead of continuing the integration, the network wrote a single specification, shaped like an internal RFC, with clinic managers, doctors, and administrators at the table. It named six use cases (book, cancel, reschedule, manage availability, override, send reminders), a domain model where a patient is unique across the whole network and a doctor's availability lives with the doctor rather than any one clinic, and explicit rules: booking up to 90 days out, specialty-specific slot lengths, reserved emergency slots, 24-hour cancellation notice. Against that one specification, they built five separate implementations, a patient app, a doctor-facing scheduling tool, a clinic admin dashboard, an integration layer, and a backend service, each suited to its own users rather than forced into one shared codebase.

REPORTED RESULT

- Four months and $1.2 million, against the vendor's 12-month, $3 million quote
- Staff time on manual scheduling reported down 40 percent
- Five new clinics added later by configuring the existing systems, not building new ones

The network's account credits the consistency to the shared specification, not to standardizing on one vendor or one stack. Five different implementations enforced the same rules because all five were built against the same document.

CASE 3 · E-COMMERCE

An e-commerce team traces its inconsistent AI output back to a missing specification

An e-commerce company that had grown from a startup to $50 million in revenue had a codebase built by different teams in different eras, each with its own patterns. When they asked AI assistants to generate features, the results were inconsistent from one call to the next, with validation logic, naming, and error handling each drawn from a different assumption every time. Developers spent more time fixing the generated code than they would have spent writing it themselves.

The cause was the absence of a shared specification for the AI to work from. Domain models lived in an outdated wiki. Architecture decisions were scattered across old chat threads, and the only place implementation conventions existed at all was in code review comments. Requirements in the issue tracker were often ambiguous too. Every prompt asked the AI to guess at validation, naming, service boundaries, error handling, and testing, and a different guess came back each time.

The team spent two weeks writing it down: a layered architecture (controllers, services, repositories, REST only, Postgres, Redis), domain models for customer, product, cart, order, payment, and inventory, and implementation conventions covering validation, testing, naming, error handling, and logging. Business rules went in too: cart expiry after 24 hours idle, purchases limited to items in stock, refunds processed within 48 hours. The next feature they asked AI to build, gift card purchasing, came back following every one of those conventions, none of which the prompt restated.

REPORTED RESULT

- Feature turnaround down from two to three weeks to three to five days
- Review time reported down 60 percent
- Sprint velocity up from 12 to 25 story points

The team's own conclusion was that AI output tracked specification clarity directly. Reviewers stopped fixing style and started checking whether the generated code matched business intent, a different job than the one they'd been doing.

CASE 4 · ENTERPRISE ARCHITECTURE

An enterprise replaces a widely ignored 150-page standard with four short protocols

A large enterprise ran dozens of internal systems, built by different teams over different years in Java, Python, Go, and Node.js. Every integration meant reconciling different error handling, different validation, and different data formats, and the architecture team spent more time mediating between teams than helping any one of them ship.

Their first attempt at fixing this was a 150-page architecture standards document and mandatory training. Teams pushed back, arguing that what worked for a payment system didn't fit a scheduling system, and mostly kept using their own patterns. The document went largely unread.

The architecture team changed what it was standardizing. Instead of dictating a technology stack, they wrote four short protocol specifications in RFC style, one each for error handling, logging, authentication, and data validation, fixing only the behavior two systems needed to agree on. Error handling could still be implemented in Java, Python, or Go. Logging could run through ELK, Splunk, or Datadog. Authentication could use Keycloak, Auth0, or Okta. Every system had to implement the four protocols; none was told how.

The reported results here are qualitative rather than numeric. Systems that previously couldn't communicate cleanly began interoperating. Teams kept full control of their own technology choices. New teams could tell what was expected of them without reading 150 pages. Debugging across systems became tractable, because error handling, logging, and validation were now consistent everywhere. Adding a new system stopped requiring an extensive architecture review.

In the architecture team's own description, its role shifted from enforcing rules to maintaining a small number of protocols. That is the same relationship that lets many independent implementations of HTTP interoperate without agreeing on anything else.

CASE 5 · PUBLIC SECTOR

A government agency modernizes benefits processing one program at a time

A government agency's benefits system, built in 1995, handled housing assistance, food programs, healthcare benefits, and education funding. Processing a single application took 45 days, with manual review at every step and constant data-entry errors. Downtime wasn't an option: hundreds of thousands of people depended on the system every day. A 2015 attempt to estimate a full rewrite came back at five years and $80 million, and the agency abandoned it as too risky.

Rather than attempt that rewrite again, the agency recovered a specification program by program: reading the existing code, interviewing the benefit officers who still understood the rules, and cross-checking both against current regulation. Each recovered specification (use cases such as apply for benefits and verify eligibility, a domain model of applicant, application, benefit, and verification) was validated with the officers before any new implementation was generated. The old and new systems ran side by side before each cutover.

Four programs went through the process in sequence: housing assistance first, in three months; food programs next, in two; healthcare benefits, the largest and most complex, in three; education funding last, in one month, to consolidate what the team had learned.

REPORTED RESULT

- All four programs done in nine months and about $4 million, against the abandoned $80 million, five-year estimate
- Housing assistance processing time down from 45 days to 5, error rate down from 8 percent to 0.3 percent
- Healthcare benefits, the largest program, reported at 15 times faster processing
- Across all four programs: average processing time down to 3 to 5 days, overall error rate down to about 0.2 percent

The agency's own account credits recovering and validating the existing rules program by program, not any single new technology, and treats each phase as independently tested before the next one began.

READING ACROSS THE FIVE CASES

What these five cases share, as this book reads them

The patterns below are this book's interpretation of the five cases in this appendix, not a claim about organizations in general. Five cases, however consistent, are not a sample a reader should generalize from.

- ☐ **Specification work is what each organization credited, not a technology choice.** The bank's account is about understanding the ledger's rules, not about Java over COBOL. The enterprise's account is about four protocols, not about which languages its systems happened to use.
- ☐ **Every organization reported spending time up front.** Eight weeks recovering the bank's rules. Two weeks writing the e-commerce team's architecture document. None reported skipping that cost, only recovering it later.
- ☐ **Once a specification existed, more than one team or technology could build against it without drifting apart.** The clinic network shipped five different implementations from one specification. The enterprise let four languages coexist under four protocols.
- ☐ **Where AI was involved, its output tracked how precisely the specification was written.** The e-commerce team reported this directly; the bank's insistence on validating against 10,000 production test cases before cutover is the same idea applied to a higher-stakes system.

Two of the five cases, the bank and the government agency, involved recovering a specification from a system nobody had fully documented, rather than writing one from a blank page. In both, the specification was treated as separate from the technology under it once it was recovered, which is why each organization reports having replaced that technology without losing the business rules.

Whether these patterns hold at your organization is a question this appendix cannot answer. It can only report what five specific organizations said happened at theirs.

BACK MATTER

Appendix B: RFCs as precedent

Three protocol specifications that have outlived implementations written against them, and what an organization can borrow from the way they were written.

Every claim in this book about specifications outliving implementations has already been tested, for longer than any of this book's examples have existed, by the documents that describe how computers talk to each other. This appendix looks at three of them: RFC 791, RFC 793, and RFC 7230. None of the three cares what language implements it. Each has outlived implementations written against it. This book borrows that pattern, and the borrowing deserves a clear look before the comparison to organizational specifications.

What an RFC refuses to say

An **RFC**, short for Request for Comments, is a numbered document describing how a piece of internet infrastructure should behave, published today through the Internet Engineering Task Force (IETF). The name undersells it. The first RFCs, starting in 1969, really were working notes soliciting comment. What the series became is something closer to law: a body of specifications that most of the internet's traffic still runs on, decades later, without anyone needing to ask permission to implement one.

An RFC that defines a protocol reads like a use case crossed with a domain model. It names the actors: a client, a server, sometimes an intermediary. It states preconditions and postconditions for every exchange between them. It enumerates the valid states a connection can be in and the transitions allowed between them. It specifies what a compliant implementation must do when something goes wrong. What it does not do, ever, is name a programming language, a data structure, or a vendor.

That omission is deliberate, and it is the whole point. An RFC describes what a protocol does. It leaves how any particular system builds that behavior entirely to whoever is implementing it. A specification that tried to pin down implementation would not travel between a mainframe and a phone; naming an implementation choice is exactly what a specification cannot survive doing, if it wants to outlive the technology fashionable the year it was written.

Three specifications, outliving everything built against them

RFC 791 defines the Internet Protocol: how a packet finds its way from one machine to another across networks neither machine controls. **RFC 793** defines the Transmission Control Protocol built on top of it: how two machines agree to open a reliable connection, keep it in order, and close it cleanly. Both were published in September 1981. They standardized work underway through the 1970s, and together they are the reason "TCP/IP" is spoken as one phrase. **RFC 7230**, published in 2014, is one part of a multi-document revision of HTTP, the protocol a browser uses to ask a server for a page.

None of the three names a technology. RFC 7230 says a server responds to a request with a status line, a set of headers, and an optional body, in a specified order, under specified conditions. It does not say which operating system runs the server, which language parses the request, or which company built either one. A server written in C in the 1990s and a server written in a language that did not exist yet when RFC 793 was published still open a TCP connection the same way, because both were built against the same document.

> **What survived, what didn't**
>
> The implementations written against RFC 791 in the early 1980s are gone: retired hardware, discontinued operating systems, companies that no longer exist. RFC 791 is still the specification a new network stack is written against today. The document outlasted every machine that ever ran code built from it.

How a protocol changes without a central rewrite

Protocols evolve, and the RFC series has a mechanism for that: a new RFC supersedes or amends an old one, and implementations update to match. TLS, short for Transport Layer Security, the protocol that secures most web traffic, went through versions 1.0, 1.1, 1.2, and 1.3 across roughly two decades. Four versions, four separate specifications. The protocol evolved through versioned documents rather than through a coordinated edit to every server that speaks it.

What that example does and doesn't show needs a precise reading. Implementations updated to support each new TLS version because the specification told them what the new version required, not because a single unedited codebase silently absorbed every change. Server software was patched, libraries were rewritten, and some old implementations were retired outright rather than upgraded. What stayed constant was the destination: every implementation, however it got there, was aiming at the same published document. That is a more modest claim than "nothing was hand-edited," and it is the one this book is willing to defend.

NOTE

The comparison this appendix draws between a domain model and a protocol, and between a business rule and a protocol constraint, is this book's own analogy. It is not a claim that internet standards bodies or software architects treat organizational specifications as RFCs today. Appendix C works the analogy through in full; here it is stated once, plainly, as an interpretation rather than an established practice.

The mapping this book draws

Line up a protocol specification next to the four-layer specification this book builds across Chapters 2 through 5, and the shapes match closely enough to be useful, even though nobody designed one to resemble the other.

A **domain model** plays the role an RFC gives its message format: both name the things a system reasons about and the relationships between them, before either says a word about storage or transport. A **use case** does the same job a protocol's state machine does, laying out a sequence of steps between named actors, a defined starting point, a defined successful outcome, and the branches that apply when something deviates from the main path. And a **business rule** reads like a protocol constraint. Both state something that must hold no matter which implementation is running, the kind of statement an RFC writes as a "MUST".

The parallel is not exact, and this book does not claim it is. A protocol coordinates strangers who will never meet and have every incentive to interoperate; an organization's specification coordinates colleagues who already share an employer and could, in principle, just ask each other. But the same discipline justifies the parallel in both places: write down what must be true, leave open how any particular system makes it true, and the specification keeps working long after the system that first satisfied it is gone.

Why most organizations never adopted this discipline

If separating what from how has worked this well for fifty years of internet infrastructure, the obvious question is why it stayed at the internet's edge instead of moving inside the organizations that build on top of it. A few reasons are worth naming, though the reading is this book's own.

Software engineering culture rewards shipped features, and an RFC-style specification looks, to a team under deadline pressure, like paperwork standing between them and the thing they're actually paid to build. The feedback loop is part of it too: the IETF can afford to spend years getting a protocol right because thousands of implementers depend on getting it right once. A single product team rarely feels that pressure from the other direction; the cost of an undocumented rule doesn't show up until the one engineer who remembers it leaves, and by then the bill has been separated from the decision that created it. Most organizations also have no equivalent of a standards process. There's no body that reviews a domain model the way a working group reviews a draft RFC, no version history a new hire can read to see how a rule changed and why.

Building that discipline inside an organization means creating what the internet's standards process already supplies: an owner for the specification, and a habit of reviewing every change before it ships. A specification has to become a document someone owns and updates, the way an editor owns a standard, instead of a comment left to decay in the code. Adopting the model costs time in the early cycles. The payoff compounds later, the way it did for the internet: it shows up on the ten-thousandth exchange, handled by an implementation whose authors never met anyone who wrote the original specification.

BACK MATTER

Appendix C: A worked organizational specification

The Riverside Clinics booking feature, specified completely enough for a team that has never read this book to implement it correctly.

Chapter 2 gave the booking feature a vision. Chapter 3 turned that vision into requirements and a use case. Chapter 4 gave the use case a domain model. Chapter 5 assembled the four layers into one document. None of those chapters wrote the specification out in the form an implementing team actually receives it: every entity's fields, the exact sequence of a booking request, the rules a database has to enforce, the notifications a patient should expect, and the values a status field is allowed to hold. This appendix writes that version.

This appendix borrows a name from the internet standards Appendix B discusses and calls a document shaped like this one an internal RFC. The borrowing is this book's analogy, not a practice organizations already follow. This book has used the word specification throughout, and keeps using it here. The two names describe the same artifact: a precise, technology-independent statement of what a system must do, versioned and owned the way a codebase currently is, implementable by a Java team, a Go team, or an AI assistant working from nothing else.

What follows: scope, the entities the feature depends on, the booking protocol step by step, the consistency rules a datastore must hold, notification behavior, the enumerations that constrain valid state, implementation notes for whoever builds it, and a proposed version 1.1 showing how the specification itself changes when the business asks for more.

Overview and scope

> **Riverside Clinics Appointment Scheduling Specification**
>
> `Version: 1.0`
>
> `Status: Active`
>
> `Owners: Clinic Operations, Patient Services Engineering`
>
> `Last revised: 2026-01-12`

This specification defines the business protocol for scheduling a patient appointment at a Riverside Clinics location. Any system that lets a patient or a staff member create, cancel, or view an appointment must conform to it, whether that system is a public booking website, a phone-based intake tool, or a front-desk terminal. The specification says nothing about programming language, deployment platform, or storage engine. Two conforming implementations may share no code at all and still interoperate, because agreement lives in the entities, the rules, and the messages passed between them.

IN SCOPE

- Patient-initiated appointment booking
- Staff-initiated appointment creation
- Appointment cancellation
- Doctor and clinic availability
- Concurrent-booking conflict resolution

OUT OF SCOPE

- Patient medical records
- Billing and insurance
- Telehealth session delivery
- Rescheduling as a separate flow (v1.0 handles a reschedule as a cancellation followed by a new booking)

Telehealth and rescheduling are visible gaps on purpose. Chapter 4 flagged telehealth as a case this specification's enumeration doesn't yet cover; closing that gap is future work this specification leaves for whoever writes the next version.

Entities: Patient, Doctor

Chapter 4 named Patient, Doctor, Clinic, Appointment, and TimeSlot as an excerpt of Riverside's domain model, sufficient to make one point about entities and value objects. A specification an implementing team can build from needs every field the protocol below actually touches. What follows extends that excerpt without contradicting it: each entity keeps the identity, attributes, and relationships chapter 4 established, with the additional fields the protocol requires.

Patient (entity)

- `patient_id`: unique identifier, immutable once assigned.
- `name`: string.
- `email`: string, must conform to the address syntax in RFC 5322.
- `phone`: string, ITU-T E.164 format.
- `date_of_birth`: date, must be in the past.
- `medical_record_number`: unique across all patients (chapter 4).
- `registration_date`: timestamp, RFC 3339 format.

Relationship: has many Appointments.

Doctor (entity)

- `doctor_id`: unique identifier.
- `name`: string.
- `specialty`: enumeration, Cardiology, Dermatology, Orthopedics, Family Medicine, Psychiatry (chapter 4).
- `years_licensed`: integer, non-negative.

Relationship: works at one or more Clinics, many-to-many (chapter 4). A booking request must therefore name which clinic a doctor is being requested at; nothing about a doctor alone tells the system where they'll be that day.

Relationship: has many UnavailabilityWindows, the value object defined below.

Entities: Clinic, Appointment

Clinic (entity)

- clinic_id: unique identifier.
- name: string.
- business_hours: the days and hours the clinic accepts appointments (chapter 4). v1.0 fixes this to Monday through Friday, 08:00 to 17:00, clinic-local time.

Relationship: employs many Doctors.

Appointment (entity)

- appointment_id: unique identifier.
- patient_id: reference to Patient.
- doctor_id: reference to Doctor.
- clinic_id: reference to Clinic. Must be one of the clinics the referenced Doctor works at.
- scheduled_slot: TimeSlot value object (chapter 4).
- duration_minutes: 15, 30, 45, or 60.
- status: enumeration, defined in full under Enumerations.
- reason_for_visit: string, must not be empty.
- confirmation_number: an opaque identifier returned to whoever booked the appointment.
- created_timestamp: RFC 3339 timestamp.
- created_by_actor: enumeration, Patient or Staff.

Relationship: involves exactly one Patient, exactly one Doctor, at exactly one Clinic.

Value objects: TimeSlot and UnavailabilityWindow

TimeSlot (value object, chapter 4)

- date, start_time, end_time: end_time is exactly 15 minutes after start_time. A longer visit occupies consecutive slots.

No identity of its own; meaningful only as an Appointment's scheduled_slot. This specification computes a slot's availability at request time rather than storing it as a separate record with its own status; see Rule 3 under Consistency and validation rules.

UnavailabilityWindow (value object)

- doctor_id: reference to Doctor.
- clinic_id: optional. Absent means the doctor is unavailable for the window's duration at every clinic where they work.
- start, end: the unavailable range, as RFC 3339 timestamps, so one window covers part of a day or several whole days.
- reason: string, for staff reference only (vacation, conference, and similar cases).

No identity of its own; two windows naming the same doctor, range, and reason are the same fact recorded twice, replaced rather than edited when a doctor's plans change.

A doctor's true availability at a given clinic is the clinic's business hours, minus any UnavailabilityWindow overlapping the slot, minus any date and time already held by a Scheduled Appointment for that doctor.

The booking protocol: requesting an appointment

A protocol message is a defined request and the defined responses it can produce. The first one collapses the display-and-select steps of chapter 3's Schedule Appointment use case into a single message every conforming system implements identically: the caller supplies a date range and the system selects the slot, so the choice a patient makes in the interface never becomes a protocol state the server has to hold.

RequestScheduleAppointment

Initiator: Patient, or Staff acting on a patient's behalf.

Parameters: patient_id, doctor_id, clinic_id, preferred_date_range_start, preferred_date_range_end.

Preconditions: the referenced Patient and Doctor exist; the Doctor works at the referenced Clinic; preferred_date_range_end is no more than 90 days from today (Rule 1).

RESPONSE: SUCCESS

AppointmentScheduled, carrying appointment_id, scheduled_slot, confirmation_number. A confirmation notification is dispatched within 60 seconds (see Notifications).

RESPONSE: ALTERNATIVE

NoAvailableSlots, carrying doctor_id, clinic_id, requested_date_range, and suggested_alternatives: other slots with the same doctor at a later date, or other doctors of the same specialty with availability inside the requested range.

RESPONSE: ERROR

ValidationError, carrying error_code and error_message. Returned when a precondition fails: an unknown patient or doctor, a doctor who does not work at the named clinic, or a date range outside the 90-day horizon.

The booking protocol: races and cancellation

Concurrent reservation

Problem: two RequestScheduleAppointment messages resolve to the same open slot before either booking completes.

1. The system re-checks availability for the exact doctor, clinic, date, and time immediately before creating the Appointment record, inside the same transaction.
2. If the slot is already held by a Scheduled appointment, the system returns NoAvailableSlots to the second request, with alternatives computed at that moment.
3. No two Scheduled appointments result for the same slot. Whether an implementation prevents the second by locking, by resolving at commit, or by holding the slot briefly is left open (chapter 10).

CancelAppointment

Initiator: the Patient who holds the appointment, or Staff on their behalf.

Parameters: appointment_id.

Preconditions: the Appointment exists and its status is Scheduled; its scheduled_slot start is at least 24 hours away.

RESPONSE: SUCCESS

AppointmentCancelled, carrying appointment_id and cancellation_timestamp. A cancellation notification is dispatched within 60 seconds.

RESPONSE: ERROR

TooLateToCancel, carrying appointment_id, hours_until_appointment, and a message stating the 24-hour notice requirement (Rule 7).

Consistency and validation rules, part one

Every implementation must enforce these regardless of storage technology. A database schema that violates one of these rules has a defect; the specification itself stays unambiguous.

RULE 1: THE 90-DAY BOOKING HORIZON

An appointment's `scheduled_slot` date cannot be more than 90 days from the request date. Doctor schedules at Riverside are only confirmed that far out; the number is the same one clinic operations agreed to in chapter 3.

RULE 2: BUSINESS HOURS AND SLOT ALIGNMENT

A `scheduled_slot` must fall inside the referenced clinic's `business_hours`. Its `start_time` must land on a 15-minute boundary, and the Appointment's `duration_minutes` must be 15, 30, 45, or 60.

RULE 3: DOCTOR CAPACITY, NO DOUBLE-BOOKING

No two Scheduled appointments for the same doctor may overlap in time, regardless of which clinic each one is at. A doctor who works two clinics still has one calendar. The many-to-many relationship in the domain model describes where a doctor can work; Rule 3 governs how many places they can be at once, and the answer is always one.

RULE 4: PATIENT UNIQUENESS

A patient cannot hold two Scheduled appointments with overlapping times.

Consistency and validation rules, part two

RULE 5: IMMUTABLE HISTORY

Once an Appointment reaches Completed or Cancelled, none of its attributes change again. The record has become history.

RULE 6: TEMPORAL CONSISTENCY

`scheduled_slot` must be at or after `created_timestamp`. An appointment cannot be booked into the past.

RULE 7: CANCELLATION NOTICE

A patient must cancel at least 24 hours before `scheduled_slot` starts. The notice period exists so the clinic has time to offer the slot to another patient.

Rules 1, 6, and 7 all depend on the system's clock agreeing with the patient's clock and with every other server's clock; the implementation notes later in this appendix say what that requires.

Notification behavior

Four events trigger a notification. Each has a defined recipient, defined contents, and a delivery timing requirement, stated below against whichever moment that particular notification is timed from.

EVENT	RECIPIENT	CONTENTS	TIMING
Appointment scheduled	Patient, email	Appointment details, confirmation number, cancellation instructions, the 24-hour cancellation notice	Within 60 seconds
Appointment cancelled	Patient, email	Appointment details, cancellation timestamp, confirmation the slot is open again	Within 60 seconds
Appointment reminder	Patient, email	Appointment details, how to cancel and rebook	24 hours before the slot, inside the hour before that mark
Appointment completed	Patient, email	Visit summary, next-step instructions	Optional, by clinic policy

Notification delivery must be at-least-once: sending the same notification twice is acceptable, since a patient who reads two identical confirmation emails loses nothing. Failing to send one is not acceptable, because a patient who never learns their appointment was cancelled will show up to a slot that no longer exists.

Enumerations

An enumeration limits an attribute to a fixed, named set of values (chapter 4). Three attributes in this specification are enumerations, and every conforming implementation must reject a value outside the set.

Appointment.status

- `Scheduled: confirmed, waiting to occur.`
- `Completed: the visit occurred.`
- `Cancelled: the patient or staff cancelled it before it occurred.`
- `NoShow: the patient did not arrive and did not cancel.`

Doctor.specialty

`Cardiology, Dermatology, Orthopedics, Family Medicine, Psychiatry (chapter 4).`

Appointment.created_by_actor

`Patient, Staff.`

Chapter 4 warned that an enumeration frozen after the first draft eventually stops covering what the business actually does. Telehealth, when Riverside adds it, will need a fifth status value or a separate visit-type attribute; this specification does not add one speculatively before the business asks for it.

Implementation notes

Guidance for whoever builds this, without mandating how they build it.

PERSISTENCE

Any store capable of enforcing Rules 1 through 7 is acceptable: a relational database, a document store, an event store. The rules constrain behavior; schema shape is left to whoever implements them.

ATOMICITY

Creating an Appointment must be one atomic operation: the record is fully written, or nothing is written. The requirement rules out a design that writes the Appointment and updates availability as two separate steps that could fail between one and the other.

IDEMPOTENCY

RequestScheduleAppointment is not idempotent by default; a client that resubmits an identical request after a timeout, without knowing whether the first one succeeded, can end up creating two appointments unless it supplies its own request identifier for the server to deduplicate against. Notification delivery is the opposite case: at-least-once, because duplicates there are harmless.

CLOCK SYNCHRONIZATION

Rules 1, 6, and 7 all compare timestamps across a request's origin and the server evaluating it. A deployment running more than one server must keep every server's clock synchronized, typically through the Network Time Protocol (NTP), or a patient whose local clock reads three minutes fast can appear to cancel with 23 hours and 57 minutes' notice instead of the 24 hours the rule requires. All timestamps in this specification follow RFC 3339, which requires an explicit UTC offset, as in `2026-07-25T14:30:00-04:00`, so every server parses the same string the same way regardless of locale.

TESTING BASELINE

- A valid RequestScheduleAppointment succeeds and produces exactly one Appointment.
- A request for a slot outside business hours, or past the 90-day horizon, returns ValidationError.
- Two simultaneous requests for the same slot: one succeeds, one receives NoAvailableSlots, and no second Appointment is created.
- Cancellation inside and outside the 24-hour window produces the correct response in each case.
- A server restart mid-transaction leaves the datastore consistent with Rule 3 and Rule 4.

Versioning: what changes in a proposed v1.1

This specification is versioned the way the rest of the book argues a specification should be: changes are proposed, reviewed, and stated as a difference against the version implementations already run. Nothing below is built. It is what the next version would say if Riverside's clinic operations team asked for it.

ADDITION: RECURRING APPOINTMENTS

A new entity, `RecurringSeries`: `series_id`, `patient_id`, `doctor_id`, `clinic_id`, `recurrence_rule` (for example, weekly for six occurrences), `series_status` (`Active`, `Cancelled`). Appointment gains an optional `series_id` reference. Creating a series generates individual Appointment records, but only for occurrences that fall inside Rule 1's 90-day horizon; later occurrences generate as the horizon rolls forward. Cancelling one occurrence cancels that Appointment alone, under Rule 7, without touching the series; cancelling the series cancels every future Scheduled occurrence at once.

ADDITION: MULTI-CLINIC BOOKING

In v1.0, RequestScheduleAppointment requires `clinic_id`; the patient already knows which location they want. Proposed v1.1 makes `clinic_id` optional. When it's absent, the system searches every clinic the named doctor works at and returns the earliest available slot across all of them. AppointmentScheduled's response then includes the `clinic_id` it selected, so the patient learns where to go from the response itself rather than from a choice they made earlier.

Both additions are backward compatible: every v1.0 field keeps its meaning, and a v1.0 request with `clinic_id` supplied behaves exactly as it does today. An implementation states which version of this specification it supports. A v1.0 client and a v1.1 client can therefore both be live at once during a migration, each conforming to the version it declares.

What this specification replaces

Nothing in this appendix depends on Java, Spring Boot, or any of the other technologies that chapter 7's skills name. A team could implement it in Go next year and a different team could reimplement it again after that, and a patient booking an appointment would notice no difference, because the protocol stayed the same while the code underneath it changed twice. That is the argument chapter 11 makes about specifications generally, applied here to one feature in full.

CITED STANDARDS

- RFC 5322, Internet Message Format: the address syntax Patient.email must conform to.
- ITU-T E.164, the international public telecommunication numbering plan: the format Patient.phone must conform to.
- RFC 3339, Date and Time on the Internet: Timestamps: the format every timestamp in this specification follows.
- ISO 8601: the broader international date-time standard RFC 3339 profiles for internet use.

Appendix B reads RFC 791, RFC 793, and RFC 7230 as the precedent this format borrows from. This appendix is the same discipline turned inward, on a business protocol instead of a network one.

BACK MATTER

Glossary

Most of these terms are defined on first use in the body of the book. This page collects those definitions in one place, in the sense the book gives them rather than any wider dictionary sense.

TERM	DEFINITION
Specification	A precise, technology-independent description of what a system must do: which actors are involved, what the rules are, and what counts as success or failure. It combines vision, requirements, use cases, and a domain model into one document an AI assistant can implement directly.
Vision	The explicit statement of why a system exists and for whom, written in business language before any requirement is captured. It is stable over time and constrains scope before any architecture decision locks anything in.
Requirement	An observable, testable statement about system behavior that stays silent on implementation. A requirement states a rule; it does not say how the system enforces it.
Use case	A complete description of one interaction: an actor, preconditions, a main scenario, alternative flows, postconditions, and the business rules that govern it. It is the unit AI implements best, because it specifies a whole interaction rather than a single property.

TERM	DEFINITION
Domain model	A description of the business objects a system runs on, and the relationships and constraints between them, independent of how anything is stored. It captures business meaning, not database design.
Entity	A thing the business cares about that has its own identity and attributes that can change over time, such as a patient or an appointment. Two entities with identical attributes are still different things if their identities differ.
Value object	An immutable concept with no identity of its own, meaningful only in the context of the entity that holds it, such as an address or a time slot. Value objects are typically attributes of entities rather than entities themselves.
Enumeration	An attribute limited to a fixed, named set of valid values, such as an appointment status of Scheduled, Completed, Cancelled, or NoShow. It is neither an entity nor a value object, and it needs revisiting whenever the business adds a case the existing values do not cover.
Architecture	The set of decisions that apply to every feature in a codebase: layering, dependency direction, naming, module boundaries, and how components communicate. Documented once, it replaces re-explaining the same structural rules in every request made to AI.
Skill	Reusable implementation knowledge for a specific technology stack: how an organization writes tests, handles errors, or structures a service. A skill is distinct from architecture and stays a versioned, living document as conventions change.
Context	The information an AI assistant already has before being asked to do something, as opposed to what a single prompt spells out at the moment of the request. This book's reading is that complete, relevant context does more for the result than careful wording does.
Protocol	A specification of what a system does, kept independent of how any one implementation builds it, the sense this book borrows from internet RFCs. A business process specified as a protocol can be implemented differently by different teams and still interoperate.
Implementation	The technology-specific realization of a specification: the working code, database schema, and configuration that make a specified behavior run. Implementation translates a decision already made; it does not make the decision.

Notes and references

The claims in this book carry different kinds of standing. The table below sorts the sources behind them by that standing: reported practice from one methodology, established internet standards, and established professional practice this book draws on rather than originates.

CLAIM TYPE	SOURCE
Reported practice	Simon Martinelli's specification-driven development methodology, credited throughout this book as the source of its core recommendations.
Standard	RFC 791, Internet Protocol
Standard	RFC 793, Transmission Control Protocol
Standard	RFC 7230, HTTP/1.1: Message Syntax and Routing
Standard	RFC 5321, Simple Mail Transfer Protocol
Standard	RFC 5322, Internet Message Format
Standard	RFC 3339, Date and Time on the Internet: Timestamps
Standard	RFC 7807, Problem Details for HTTP APIs
Standard	ISO 8601, date and time representation
Standard	ITU-T E.164, the international public telecommunication numbering plan

CLAIM TYPE	SOURCE
Established practice	Eric Evans, domain-driven design
Established practice	The Agile Manifesto
Established practice	Architecture decision records
Established practice	arc42
Established practice	The C4 model
Established practice	The Model Context Protocol
Established practice	Martin Fowler's Strangler Fig pattern

NOTE

The case studies in Appendix A are presented as reported by the organizations involved, with names and identifying details changed. This book does not carry forward the aggregate percentage-range statistics that appeared in the original source manuscript's packaging material, because those ranges do not trace back to any individual case's figures. Where a number appears in this book, it is attributed to the single case that reported it.

Index

Page numbers are assigned after final pagination and are not yet included. Entries below are grouped by the four categories this book uses, not alphabetized within them.

Concepts

Artifacts and templates

Examples

- Riverside Clinics appointment scheduling (primary running example)
- The bank's core ledger migration (variant example)
- The Schedule Appointment use case
- The Riverside Clinics domain model: patients, doctors, appointments
- A Spring Boot implementation skill for Riverside Clinics
- Two teams under the same deadline, one prepared and one improvising
- Three engineers, three valid implementations of one use case
- The bank's ledger specification, carried across three technology generations

Risks and failure modes

- Asking AI to guess business intent from a one-line request
- Domain entities and attributes invented without a model
- A vision statement that only restates the feature request
- Large, all-purpose instruction files instead of small, specific skills
- Legacy business rules that survive only as code, not documentation
- AI rewriting an undocumented system on its own authority
- Straight line-for-line migration instead of recover, validate, improve, generate
- Aggregate percentage claims that do not trace back to individual cases
- Review effort scaled to code volume instead of business risk